

01_EGAL_25.x_Communication _Architecture_ADR¹.md

PART I — CONTEXT & BACKGROUND

1.1 Historical Context

From **EGAL-24.x.x** → **EGAL-25.x.x**

The evolution toward **EGAL-25.x.x** begins after the successful stabilization of:

None

EGAL-24.x.x

Elder Guidance & Accessibility Layer

for the myClan genealogy platform.

EGAL-24 successfully established:

- elder-friendly onboarding
- low-anxiety interaction model
- voice-supported accessibility
- submit-gated validation
- controlled attention routing
- browser-safe voice behavior

¹ Architecture Decision Record (ADR) - Hồ sơ quyết định kiến trúc

- accessibility-first UX for elderly users

The fundamental doctrine of EGAL-24 can be summarized as:

None

Technology must adapt to elderly users,

not the opposite.

This architectural direction proved successful during UAT across:

- LoginForm
- RegisterForm
- JoinClanForm
- CreateClanForm
- ForgotPasswordForm
- VerifyResetCodeForm
- ChangePasswordForm
- WaitingPage

As a result:

None

EGAL-24 Elder Support Doctrine

is considered production-safe.

However, a new problem emerged.

The platform achieved:

None

elder-friendly onboarding

but communication between:

None

system ↔ user

remained heavily dependent on:

None

email only

This creates significant usability friction.

1.2 Business Reality of myClan

The myClan platform serves a unique user demographic.

Unlike conventional productivity platforms, myClan operates around:

None

family genealogy

clan management

multi-generation participation

This means the platform simultaneously serves:

Elderly Vietnamese users

Characteristics:

- often unfamiliar with email
 - rarely check inboxes
 - strongly prefer Zalo
 - lower tolerance for technical complexity
 - high anxiety toward unfamiliar UI
-

Younger Vietnamese users

Characteristics:

- comfortable with Telegram
 - comfortable with email
 - open to web notifications
 - more technically familiar
-

International users

Characteristics:

- WhatsApp-heavy
 - email-oriented
 - cross-country communication needs
-

Clan Administrators

Characteristics:

- responsible for onboarding
- approval workflow owners
- account recovery assistance
- communication coordinators

This diversity creates an unavoidable architectural challenge:

None

`one communication channel`

`cannot satisfy all users.`

1.3 Limitation of Current Architecture

The existing EGAL-24 communication model assumes:

None

`email = communication`

This assumption is insufficient.

Problems observed:

Problem A — Elderly users ignore email

For many elderly Vietnamese users:

None

`email exists only for registration.`

In practice:

- inbox is rarely checked
- password recovery becomes difficult
- approval notifications are missed
- onboarding confidence decreases

This directly contradicts:

None

Elder Support Doctrine

because:

None

supportive onboarding

without supportive communication

remains incomplete.

Problem B — Communication friction

Current business flows require communication events such as:

None

approval notice

password reset

support requests

status update

invitation

review reminder

Under email-only architecture:

many users:

None

do not receive information

at the right time

through preferred channels.

Problem C — Overdependence on a single medium

Email-only architecture introduces:

None

single-point communication dependency

Meaning:

If users do not engage with email:

None

system-user relationship weakens.

This becomes especially problematic for:

None

elderly clan members

whose primary digital communication habit is:

None

Zalo

rather than email.

Problem D — International scalability issue

myClan is intended to support:

None

Vietnamese users

+

international family members

A single communication strategy:

None

email only

cannot adequately support:

None

Telegram-oriented users

WhatsApp-oriented users

across countries.

1.4 Strategic Constraint

The solution space is constrained by several realities.

Constraint #1 — Free-first principle

Recurring operational costs must remain low.

The architecture should prioritize:

None

free

or near-free communication channels

before paid services.

This excludes:

None

SMS-first architecture

despite SMS being technically reliable.

SMS may remain:

None

future optional layer

but SHALL NOT become:

None

core dependency

for EGAL-25.

Constraint #2 — No Zalo OA dependency

The project intentionally avoids:

None

Zalo Official Account (OA)

Reasons:

- operational complexity
- policy dependency
- platform governance risk
- long-term vendor coupling

Instead:

the system assumes:

None

Personal Zalo (PZ)

for both:

None

Clan Admin

+

end user

This creates limitations:

because Personal Zalo lacks:

None

stable official automation API

Constraint #3 — Elderly-friendly onboarding

The communication solution must preserve:

None

low anxiety

low friction

The platform SHALL NOT require:

None

complex social account binding

mandatory bot setup

multi-step technical onboarding

during registration.

The onboarding philosophy remains:

None

simple first

optional later

Constraint #4 — Security preservation

Communication flexibility SHALL NOT compromise:

None

identity assurance

account recovery

auditability

Higher privilege users:

None

USER

EDITOR

ADMIN

still require:

None

reliable official communication

This strongly favors:

None

email

as a mandatory channel.

1.5 Emerging Architectural Insight

During exploratory dialogue and business analysis, a major realization emerged.

The original assumption was:

None

social channel

=

communication integration

This led toward:

None

social API coupling

However, this direction proved:

None

fragile

costly

platform-dependent

elder-unfriendly

A new architectural insight replaced this assumption:

None

social platforms

should not become system dependencies.

Instead:

they should become:

None

communication preferences

owned by users.

This insight becomes the foundation of:

None

EGAL-25.x.x

Unified Communication Architecture

PART II — PROBLEM STATEMENT

2.1 Core Problem

The current architecture lacks:

None

multi-channel communication orchestration

while serving:

None

heterogeneous user groups

with radically different:

None

technology habits

communication preferences

accessibility needs

This creates a mismatch between:

None

system communication behavior

and:

None

real-world user behavior.

2.2 Architectural Problem Categories

Problem Category A

Communication Is Hardcoded

Current architecture effectively assumes:

None

Email = communication system

This hardcoding causes:

- poor elderly adoption
- missed notifications
- low engagement
- fragile onboarding continuity

The system lacks:

None

`channel abstraction`

and:

None

`communication flexibility.`

Problem Category B

Elderly Communication Gap

EGAL-24 successfully solved:

None

`elder interaction`

but not:

None

`elder communication.`

This creates an architectural inconsistency.

The system speaks:

None

elder language

during onboarding,

but reverts to:

None

technical communication assumptions

after onboarding.

This contradiction weakens:

None

elder confidence

in the platform.

Problem Category C

Social Platform Dependency Risk

An obvious solution would be:

None

integrate Zalo

`integrate Telegram`

`integrate WhatsApp`

as:

None

`identity providers`

or:

None

`full communication APIs.`

However this introduces:

vendor lock-in

API instability

platform governance dependency

onboarding complexity

security fragility

Especially problematic:

None

`Personal Zalo`

because:

official automation support is limited.

This makes:

None

`social-as-core-infrastructure`

a poor long-term architectural choice.

Problem Category D

Missing Notification Governance

Business events currently lack:

None

`notification governance`

There is no centralized logic deciding:

None

`who gets notified`

`through which channel`

`at what reliability level`

`with what priority`

This creates:

None

scattered communication behavior

across modules.

Examples:

None

RegisterForm

Approval workflow

Password reset

Security warning

Role change

currently lack a unified notification doctrine.

Problem Category E

Role vs Accessibility Confusion

A misleading simplification initially emerged:

None

elderly users = VIEWER role

Although practical in many cases:

this assumption is architecturally incorrect.

Possible real-world cases:

None

elder clan leader

elder historian

elder editor

This means:

None

elder support

cannot depend on:

None

authorization role.

Accessibility needs must remain:

None

independent

from:

None

permission structure.

2.3 Design Requirements

EGAL-25 therefore requires:

Requirement #1

Support:

None

`multiple communication channels`

without:

None

`tight platform coupling.`

Requirement #2

Preserve:

None

`elder-friendly simplicity.`

No forced technical setup.

Requirement #3

Support:

None

free-first operation.

Requirement #4

Maintain:

None

official communication reliability

for privileged users.

Requirement #5

Introduce:

None

notification orchestration

as centralized infrastructure.

Requirement #6

Preserve:

None

EGAL-24 doctrines

especially:

None

low anxiety

low friction

controlled complexity

PART III — ARCHITECTURAL DECISION

3.1 Final Decision

EGAL-25 SHALL introduce:

None

Unified Communication Layer (UCL)

based on:

None

communication preference

rather than:

None

social platform integration

The architecture SHALL separate:

None

authentication

authorization

communication

notification delivery

into independent layers.

3.2 Foundational Decision

The system SHALL NOT treat:

None

social applications

as:

None

identity providers

Instead:

social platforms SHALL be treated as:

None

communication preference channels

only.

This is the most important architectural decision of EGAL-25.

Core philosophy:

None

Identity is NOT communication.

Communication is NOT authentication.

3.3 Communication Model

The system SHALL support:

None

multi-channel communication

using:

None

adapter architecture

Supported channels:

None

IN_APP

EMAIL

ZALO_PZ

TELEGRAM

WHATSAPP

WEB_PUSH

Each channel SHALL act as:

None

delivery adapter

instead of:

None

business logic owner.

Meaning:

business flows SHALL NOT directly call:

None

Telegram

Email

Zalo

Instead:

business flows emit:

None

notification intent

which becomes centrally orchestrated.

Example:

None

USER_APPROVED

↓

None

Notification Policy Engine

↓

None

select channels

↓

None

deliver

3.4 Zalo Decision

The project explicitly rejects:

None

Zalo OA dependency

and:

None

unofficial Personal Zalo automation APIs

for production core.

Instead:

Personal Zalo SHALL become:

None

Manual-Assisted Communication Channel

Meaning:

the platform SHALL support:

- QR connection
- guided onboarding
- deep link assistance

- pre-generated messages
- copy-to-send workflow
- manual notification queue

But SHALL NOT rely on:

None

`unofficial bot automation`

as production infrastructure.

3.5 Deferred Communication Binding Decision

Users SHALL NOT be forced to:

None

`bind communication channels`

during registration.

Instead:

the system stores:

None

`communication preference`

first.

Binding occurs:

None

later

if needed.

This doctrine minimizes:

None

elder anxiety

onboarding friction

while preserving:

None

communication flexibility.

PART IV — CORE PRINCIPLES & DOCTRINES

4.1 Purpose of This Section

This section formalizes the:

None

non-negotiable architectural principles

for:

None

EGAL-25.x.x

Unified Communication Architecture

These principles function as:

None

architecture doctrine

similar to:

None

Elder Support Doctrine

in EGAL-24.

All future implementation work:

None

must comply

with these principles unless an explicit ADR revision occurs.

4.2 PRINCIPLE #1

Identity is NOT Communication

Decision

Authentication identity SHALL remain independent from communication systems.

Meaning:

None

who the user is

must remain separate from:

None

how the system communicates

with that user.

Rationale

Many systems mistakenly couple:

None

identity

with:

None

social platform accounts

Examples:

None

Login with Facebook

Login with Telegram

Login with WhatsApp

This introduces:

- vendor lock-in
- API fragility
- onboarding complexity
- migration difficulty
- platform governance dependency

Such dependency is unacceptable for:

None

long-lived genealogy systems

because:

family platforms must survive:

None

decades

rather than:

None

social platform trends.

EGAL-25 Decision

myClan SHALL authenticate users using:

None

phone

email

password

future MFA

NOT:

None

social identity.

Examples:

Allowed

None

phone + password

email + password

Forbidden

None

login via Zalo

login via Telegram

login via WhatsApp

unless explicitly approved in:

None

future ADR.

Doctrine Statement

None

Identity must survive

outside communication platforms.

4.3 PRINCIPLE #2

Communication is NOT Authentication

Decision

Communication channels SHALL NOT become:

None

security dependencies.

Social platforms exist for:

None

communication convenience

NOT:

None

identity assurance.

Rationale

Communication platforms are:

None

unstable

from a system-design perspective.

Users may:

- uninstall apps
- change phone numbers

- stop using Telegram
- abandon WhatsApp
- lose Zalo access

Yet:

None

`system identity`

must remain stable.

If communication becomes authentication:

the system becomes:

None

`fragile.`

Example

Wrong Design

None

`User must connect Zalo`

`before account activation.`

Problem:

If Zalo changes:

None

account access breaks.

Correct Design

None

User chooses Zalo

as preferred communication.

But:

None

account remains independent.

Doctrine Statement

None

Communication may change.

Identity must remain stable.

4.4 PRINCIPLE #3

In-App Notification Is Canonical

Decision

Every notification SHALL first exist:

None

`inside myClan`

before:

None

`external delivery.`

This means:

None

`in-app notification`

becomes:

None

`source of truth.`

Rationale

External channels can fail.

Examples:

None

email not opened

Telegram unavailable

Zalo not connected

WhatsApp inactive

push disabled

However:

the platform must preserve:

None

communication auditability.

Thus:

all business communication SHALL first be:

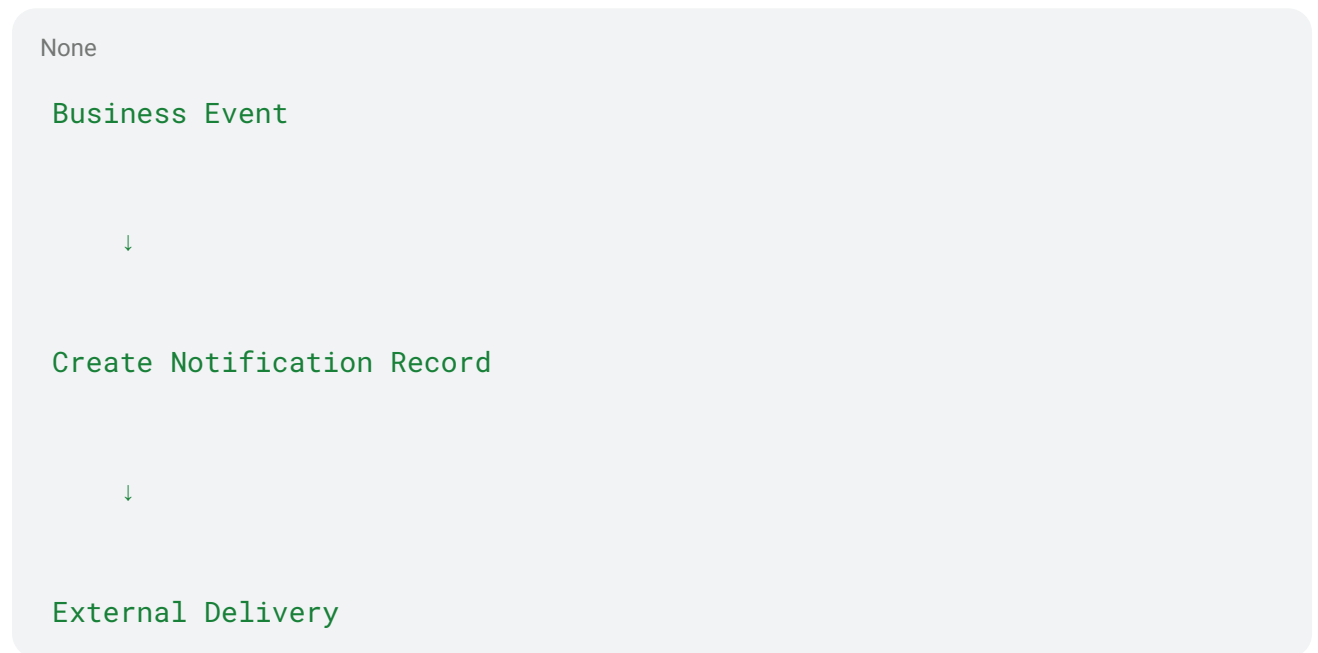
None

stored

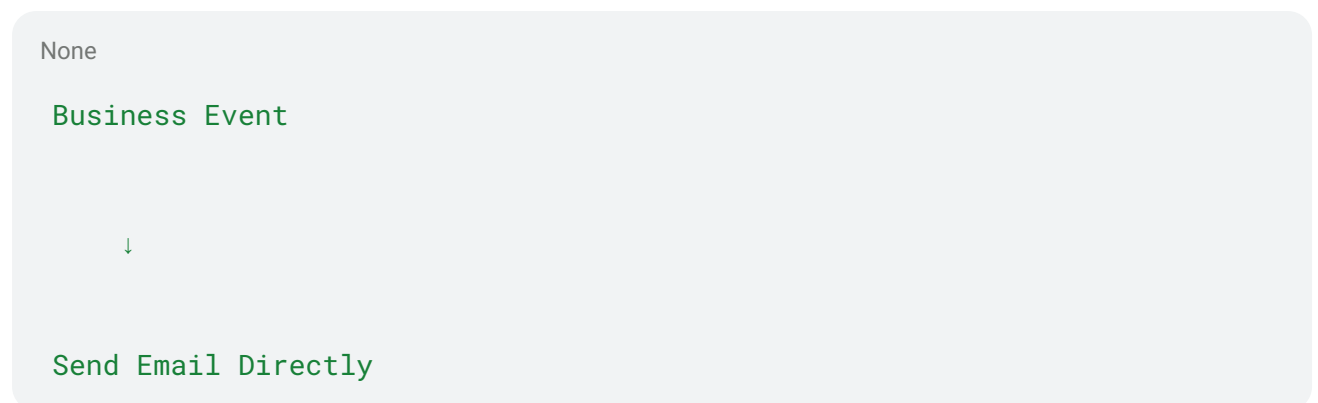
in system database.

Canonical Flow

Architecture:



NOT:



Benefits

Reliability

Message history survives.

Auditability

Admin can inspect:

None

who was notified

when

through which channel

delivery result

Recovery

Failed delivery can be retried.

Multi-channel flexibility

One notification:

None

many channels.

Doctrine Statement

None

External channels deliver.

myClan remembers.

4.5 PRINCIPLE #4

Elder Support ≠ Authorization Role

Decision

Elder accessibility SHALL remain:

None

independent

from:

None

permission model.

Historical Misconception

An early assumption suggested:

None

`elder users = VIEWER role`

Although practical in many situations:

this assumption is:

None

`architecturally wrong.`

Real-World Cases

Possible scenarios:

Case A

None

`elder clan leader`

Role:

None

`EDITOR`

`ADMIN`

Needs:

None

voice support

simple UI

guided interaction

Case B

None

young viewer

Role:

None

VIEWER

Needs:

None

standard UX.

EGAL-25 Decision

Accessibility profile SHALL be separated.

Example model:

None

`uiMode:`

STANDARD

ELDER

`needsVoiceSupport:`

`true/false`

NOT:

None

`role = elderly.`

Benefits

Architectural cleanliness

Roles remain:

None

`security-oriented.`

Accessibility remains:

None

user-oriented.

Future flexibility

Supports:

None

elder editor

disabled user

vision-impaired user

without permission redesign.

Doctrine Statement

None

Accessibility is profile.

Authorization is privilege.

4.6 PRINCIPLE #5

Email Remains Official Communication Channel

Decision

For privileged users:

None

USER

EDITOR

ADMIN

Email SHALL remain:

None

mandatory.

Rationale

Email provides:

Stability

Independent from social trends.

International standardization

Works globally.

Password recovery

Reliable reset channel.

Audit trail

Persistent history.

Vendor independence

SMTP remains universal.

Communication Model

For privileged roles:

Architecture becomes:

None

Email

+

optional social channels

+

in-app

NOT:

None

social-only communication.

Exception

VIEWER role MAY use:

None

phone + password

without mandatory email.

Reason:

None

lower risk

elder friendliness

Doctrine Statement

None

Privilege requires reliability.

Reliability requires email.

4.7 PRINCIPLE #6

Zalo Personal = Manual-Assisted Communication

Decision

Personal Zalo SHALL be treated as:

None

Manual-Assisted Channel

NOT:

None

automated infrastructure.

Rationale

The project intentionally rejects:

None

Zalo OA dependency

while acknowledging:

None

Personal Zalo

as:

None

elder-friendly communication.

However:

Personal Zalo lacks:

None

stable official automation API.

Unofficial APIs are:

None

production risk.

EGAL-25 Strategy

The system SHALL support:

QR onboarding

None

Scan to connect with Clan Admin

Deep-link assistance

None

Open Zalo

Prefilled messages

None

prepared communication templates

Manual queue

Admin dashboard:

None

messages awaiting manual send

Send assistance

None

copy-to-send

workflow.

Forbidden

EGAL-25 SHALL NOT depend on:

None

personal bot automation

reverse-engineered APIs

for production continuity.

Doctrine Statement

None

Zalo supports humans.

myClan owns workflow.

4.8 PRINCIPLE #7

Deferred Communication Binding

Decision

Users SHALL NOT be forced to:

None

connect channels

during registration.

Instead:

the system first records:

None

communication preference.

Binding may happen:

None

later.

Rationale

Complex onboarding causes:

None

elder anxiety

drop-off

confusion

Especially harmful:

None

multi-step setup.

Correct Flow

Registration:

None

Preferred channel:

Zalo

Later:

None

Would you like

to connect with Clan Admin on Zalo?

Optional:

None

Scan QR

Doctrine Statement

None

Simple first.

Optional later.

PART IV — FINAL PRINCIPLE

SUMMARY

EGAL-25 freezes the following doctrines:

None

D1 Identity ≠ Communication

D2 Communication ≠ Authentication

D3 In-App Notification = Canonical

D4 Elder Support ≠ Role

D5 Email = Official Channel

D6 Zalo PZ = Manual-Assisted

D7 Deferred Communication Binding

These principles SHALL govern:

None

all future implementation

for:

None

EGAL-25.x.x

Unified Communication Architecture.

PART V — USER SEGMENTATION MODEL

5.1 Purpose of User Segmentation

The communication architecture of:

None

EGAL-25.x.x

must acknowledge a fundamental reality:

None

not all users communicate the same way.

Unlike generic SaaS systems, myClan serves:

None

multi-generational

multi-cultural

multi-technology users.

This creates:

None

communication diversity.

A single communication strategy:

None

cannot adequately support

all user groups.

Therefore:

EGAL-25 introduces:

None

User Segmentation Model

to guide:

- onboarding
- communication routing
- UI assistance
- notification behavior
- future automation decisions

This segmentation is:

None

behavioral

NOT:

None

`security-oriented.`

Meaning:

segmentation SHALL NOT replace:

None

`authorization roles.`

Instead:

it complements:

None

`communication experience.`

5.2 Foundational Principle

EGAL-25 explicitly rejects:

None

`one-size-fits-all communication.`

Instead:

communication SHALL become:

None

`segment-aware.`

This means:

the system SHOULD adapt to:

None

`user communication habits`

rather than forcing users to adapt to:

None

`technical assumptions.`

This philosophy directly extends:

None

`EGAL-24 Elder Support Doctrine`

into:

None

`Communication Experience Design.`

5.3 Segment A — Elderly Vietnamese Users

Definition

Users characterized by:

None

higher age

low technical familiarity

high preference for simplicity

This segment typically includes:

- elderly clan members
- retired family members
- family historians
- elderly clan leaders

Important:

This segment:

None

DOES NOT imply VIEWER role.

An elderly user MAY still be:

None

EDITOR

ADMIN

Communication Characteristics

Common patterns:

High preference for:

None

Zalo

phone communication

human assistance

Low engagement with:

None

email

complex setup

multi-step onboarding

Low tolerance for:

None

technical terminology

verification complexity

Recommended Communication Strategy

Priority:

None

ZALO_PZ

↓

IN_APP

↓

EMAIL (if required)

UX Strategy

Recommended behaviors:

Simplicity-first UI

Avoid:

None

too many choices

Elder support mode

Enable:

None

voice guidance

large buttons

reassurance wording

Deferred setup

Avoid:

None

mandatory social connection

during onboarding.

Instead:

None

simple registration first

then:

None

`optional connection later.`

Communication Examples

Approval message

Preferred:

None

`in-app`

`+`

`optional Zalo assistance`

Password recovery

Recommended:

None

`email if available`

`in-app`

`manual assistance`

NOT:

None

mandatory Telegram

mandatory WhatsApp

Doctrine Statement

None

Reduce anxiety first.

Improve communication second.

5.4 Segment B — Younger Vietnamese Users

Definition

Users characterized by:

None

moderate to high technical familiarity.

Examples:

- younger family members
- contributors
- editors
- genealogy enthusiasts

Communication Characteristics

Typically comfortable with:

None

Telegram

Email

Web Push

Moderately comfortable with:

None

Zalo

Behavioral Traits

Users in this segment often:

None

check notifications frequently

accept automation

prefer speed

Recommended Communication Strategy

Priority:

None

TELEGRAM

↓

EMAIL

↓

WEB_PUSH

↓

IN_APP

UX Strategy

Recommended:

None

faster onboarding

more automation

less explanation

Example Communication

Password reset

Preferred:

None

EMAIL

+

TELEGRAM

Approval

Preferred:

None

IN_APP

+

EMAIL

+

OPTIONAL TELEGRAM

Doctrine Statement

None

Automation improves experience.

5.5 Segment C — International Users

Definition

Users living outside Vietnam or preferring:

None

international communication platforms.

Examples:

- overseas relatives
- foreign spouses
- international contributors

Communication Characteristics

High familiarity with:

None

WhatsApp

Email

Moderate familiarity with:

None

Telegram

Low familiarity with:

None

Zalo

Recommended Communication Strategy

Priority:

None

WHATSAPP

↓

EMAIL

↓

IN_APP

Constraints

WhatsApp automation SHALL remain:

None

secondary priority

for EGAL-25.x.x

because of:

- operational cost
- Meta platform dependency
- business API restrictions

Doctrine Statement

None

International reach

requires international channels.

5.6 Segment D — Clan Administrators

Definition

Users responsible for:

None

approval

moderation

support

communication

workflow execution

Communication Characteristics

Admins require:

None

reliability

auditability

consistency

over:

None

convenience only.

Mandatory Channels

Required:

None

EMAIL

Recommended:

None

TELEGRAM

Optional:

None

ZALO_PZ

Reason

Admins own:

None

account recovery

member onboarding

approval communication

Thus:

communication failure becomes:

None

operational risk.

Doctrine Statement

None

Admin communication

must prioritize reliability.

5.7 Accessibility Profile Overlay

User segmentation SHALL NOT override:

None

accessibility profile.

Accessibility remains:

None

orthogonal

to segmentation.

Example:

Elderly Admin

Segment:

None

Clan Administrator

Accessibility:

None

ELDER_MODE

VOICE_SUPPORT

Young Viewer

Segment:

None

Young Vietnamese

Accessibility:

None

STANDARD_MODE

Recommended Profile Model

Example:

None

uiMode:

STANDARD

ELDER

`needsVoiceSupport:`

`true/false`

`preferredReadingSpeed:`

`NORMAL`

`SLOW`

This preserves:

None

`clean separation`

between:

None

`authorization`

`segmentation`

`accessibility.`

5.8 Segment-Aware Communication Routing

EGAL-25 SHALL support:

None

segment-aware routing.

Meaning:

communication SHOULD adapt to:

None

real user behavior.

Examples:

Elder Viewer

None

ZALO

+

IN_APP

Young Editor

None

EMAIL

+

TELEGRAM

+

WEB_PUSH

International Member

None

EMAIL

+

WHATSAPP

Admin

None

EMAIL mandatory

PART V — FINAL SEGMENTATION

SUMMARY

EGAL-25 recognizes:

None

Segment A

Elderly Vietnamese

Segment B

Younger Vietnamese

Segment C

International Users

Segment D

Clan Administrators

Communication SHALL become:

None

segment-aware

while preserving:

None

role independence

accessibility independence

Core philosophy:

None

People communicate differently.

The system must adapt.

PART VI — ROLE MODEL & COMMUNICATION REQUIREMENTS

6.1 Purpose of Role Model

The communication architecture of:

None

EGAL-25.x.x

must preserve a critical separation between:

None

authorization

and:

None

communication behavior.

Roles determine:

None

what users are allowed to do.

Roles SHALL NOT determine:

None

how users prefer communication.

Nor SHALL roles determine:

None

accessibility needs.

This separation prevents:

None

security confusion

architectural coupling

future rigidity.

6.2 Architectural Principle

Role model exists for:

None

authorization

risk control

workflow governance

NOT:

None

communication preference.

Meaning:

Two users with:

None

same role

may still require:

None

different communication models.

Example:

USER A

Role:

None

VIEWER

Communication:

None

Zalo

Accessibility:

None

ELDER_MODE

USER B

Role:

None

VIEWER

Communication:

None

Telegram

Email

Accessibility:

None

STANDARD_MODE

Both remain:

None

VIEWER

But experience differs.

6.3 Role Hierarchy

EGAL-25 formally recognizes:

None

VIEWER

USER

EDITOR

ADMIN

This hierarchy reflects:

None

risk

authority

workflow responsibility.

6.4 VIEWER Role

Purpose

VIEWER is designed for:

None

read-only participation.

Typical examples:

- elderly family members
- passive clan participants
- relatives who mainly observe family history

Important:

None

VIEWER

≠

elderly

A VIEWER may be:

None

young

middle-aged

elderly

Risk Profile

Risk level:

None

LOW

VIEWER users:

- cannot modify critical clan data
- do not execute sensitive workflows
- have minimal operational impact

Therefore:

communication reliability MAY be:

None

`best-effort.`

Authentication Requirements

Allowed:

None

`phone + password`

Optional:

None

`email`

Future optional:

None

`lightweight OTP`

Forbidden:

None

`mandatory social verification`

`mandatory email`

Reason:

None

elder friendliness

low onboarding friction

Communication Requirements

Recommended channels:

None

ZALO_PZ

IN_APP

EMAIL (optional)

Communication style:

None

supportive

simple

low-pressure

Notification Reliability Requirement

Minimum:

None

LOW → MEDIUM

Examples:

Approval notice

Allowed:

None

IN_APP

+

optional Zalo assistance

Password reset

Preferred:

None

EMAIL if available

IN_APP

Fallback:

None

manual assistance

Doctrine Statement

None

VIEWER prioritizes simplicity over rigor.

6.5 USER Role

Purpose

USER represents:

None

active participants.

Users may:

- contribute information
 - participate in workflows
 - update family records
 - interact meaningfully with system functions
-

Risk Profile

Risk level:

None

MEDIUM

Incorrect actions MAY affect:

None

data quality

workflow consistency

Thus:

communication reliability becomes:

None

more important.

Authentication Requirements

Mandatory:

None

email + password

Recommended:

None

phone number

Future:

None

MFA optional

Reason:

Email provides:

None

recoverability

auditability

stability

Communication Requirements

Mandatory:

None

EMAIL

Recommended:

None

IN_APP

Optional:

None

TELEGRAM

WHATSAPP

ZALO

WEB_PUSH

Architecture:

None

EMAIL mandatory

+

preferred channels

Notification Reliability Requirement

Minimum:

None

HIGH

Examples:

Password reset

Required:

None

EMAIL

Optional:

None

Telegram

Approval notification

Recommended:

None

EMAIL

+

IN_APP

Doctrine Statement

None

Participation requires accountability.

Accountability requires reliability.

6.6 EDITOR Role

Purpose

EDITOR represents:

None

trusted contributors

who may:

- edit clan records
- curate genealogy data
- maintain historical information
- manage collaborative workflows

Risk Profile

Risk level:

None

HIGH

Because actions may:

None

change clan history

impact many members

Authentication Requirements

Mandatory:

None

email + password

Recommended:

None

phone number

Future:

None

MFA strongly recommended

Communication Requirements

Mandatory:

None

EMAIL

Recommended:

None

TELEGRAM

IN_APP

Optional:

None

WEB_PUSH

ZALO

WHATSAPP

Notification Reliability Requirement

Minimum:

None

HIGH

Critical events:

None

EMAIL mandatory

Examples:

- role changes
 - editor approvals
 - content conflict
 - security warning
-

Doctrine Statement

None

Data authority requires stronger communication assurance.

6.7 ADMIN Role

Purpose

ADMIN represents:

None

operational authority.

Admins own:

- onboarding approval
 - moderation
 - communication governance
 - support workflow
 - recovery assistance
-

Risk Profile

Risk level:

None

VERY HIGH

Admin communication failure MAY cause:

None

workflow disruption

support failure

approval bottleneck

Authentication Requirements

Mandatory:

None

email + password

Recommended:

None

phone number

Strongly recommended:

None

future MFA

Communication Requirements

Mandatory:

None

EMAIL

Strongly recommended:

None

TELEGRAM

Optional:

None

ZALO_PZ

WEB_PUSH

WHATSAPP

Notification Reliability Requirement

Minimum:

None

VERY HIGH

Examples:

Security warning

Required:

None

EMAIL mandatory

Member approval pending

Recommended:

None

EMAIL

+

TELEGRAM

+

IN_APP

Password reset

Required:

None

EMAIL

Doctrine Statement

None

Administrative authority requires communication certainty.

6.8 Role-to-Communication Matrix

R
o
l
e

E
m
a
i
l

Z
a
l
o

T
e
l
e
g
r
a
m

W
h
a
t
s
A
p
p

I
n
-
A
p
p

R
e
l
i
a
b
i
l
i
t
y

V
l
E
W
E
R

O
p
t
i
o
n
a
l

P
r
e
f
e
r
r
e
d

O
p
t
i
o
n
a
l

O
p
t
i
o
n
a
l

Y
e
s

L
O
W
-
M
E
D
I
U
M

U
S
E
R

M
a
n
d
a
t
o
r
y

O
p
t
i
o
n
a
l

R
e
c
o
m
m
e
n
d

O
p
t
i
o
n
a
l

Y
e
s

H
l
G
H

e
d

E	M	O	R	O	Y	H
D	a	p	e	p	e	I
I	n	t	c	t	s	G
T	d	i	o	i		H
O	a	o	m	o		
R	t	n	m	n		
	o	a	e	a		
	r	I	n	I		
	y		d			
			e			
			d			

A	M	O	S	O	Y	V
D	a	p	t	p	e	E
M	n	t	r	t	s	R
I	d	i	o	i		Y
N	a	o	n	o		H
	t	n	g	n		I
	o	a	I	a		G
	r	I	y	I		H
	y		R			
			e			
			c			
			o			
			m			
			m			
			e			
			n			
			d			

6.9 Role Model Doctrine Summary

EGAL-25 freezes:

R1

None

Role governs privilege.

R2

None

Role does NOT govern communication preference.

R3

None

Role does NOT govern accessibility profile.

R4

None

Higher privilege

=

higher communication reliability.

R5

None

Privileged users require email.

R6

None

VIEWER prioritizes simplicity.

R7

None

ADMIN prioritizes certainty.

PART VI — FINAL ROLE MODEL

PHILOSOPHY

Core philosophy:

None

Authorization determines power.

Communication determines reach.

Accessibility determines experience.

These SHALL remain:

None

independent layers

within:

None

EGAL-25.x.x

Unified Communication Architecture.

PART VII — CHANNEL STRATEGY & COMMUNICATION MODEL

7.1 Purpose of Channel Strategy

The communication architecture of:

None

EGAL-25.x.x

must solve a central problem:

None

different users

communicate differently.

Therefore:

communication SHALL NOT depend on:

None

single-channel assumption.

Instead:

EGAL-25 introduces:

None

Multi-Channel Communication Strategy

based on:

None

reliability

segment preference

role requirement

operational practicality

The system SHALL prioritize:

None

practicality over theoretical purity.

Meaning:

channels are selected because:

None

real users actually use them.

NOT because:

None

technology trend recommends them.

7.2 Communication Layer

Philosophy

EGAL-25 treats communication channels as:

None

`delivery adapters`

NOT:

None

`business logic owners.`

Meaning:

Business flows SHALL NOT directly call:

None

`Email`

`Telegram`

`WhatsApp`

`Zalo`

Instead:

all business events MUST enter:

None

Notification Orchestrator

Architecture:

None

Business Event



Notification Intent



Notification Policy Engine



Channel Selection



Delivery Adapter



Logging & Audit

This architecture prevents:

None

`tight coupling`

between:

None

`business logic`

and:

None

`communication technology.`

7.3 Supported Channel Categories

EGAL-25 formally recognizes:

None

`IN_APP`

`EMAIL`

`ZALO_PZ`

`TELEGRAM`

`WHATSAPP`

WEB_PUSH

These channels belong to:

three different classes.

Class A — Canonical Channels

Channels that are:

None

owned by myClan.

These become:

None

source of truth.

Included

None

IN_APP

Characteristics

Advantages:

- platform-controlled
- fully auditable
- persistent
- reliable
- retryable

Limitations:

None

`requires user login.`

Doctrine

None

`Every notification`

`must first exist`

`inside myClan.`

Class B — Official Communication Channels

Channels with:

None

`high reliability.`

Included

None

EMAIL

Characteristics

Advantages:

- stable
- globally supported
- password recovery friendly
- auditable
- API-independent

Limitations:

None

elder users rarely check email.

Doctrine

None

Email is official communication.

Especially for:

None

USER

EDITOR

ADMIN

Class C — Convenience Channels

Channels selected because:

None

users naturally prefer them.

Included

None

ZALO_PZ

TELEGRAM

WHATSAPP

WEB_PUSH

These channels exist to:

None

reduce friction

increase responsiveness

improve engagement.

NOT:

None

replace identity

replace official communication.

7.4 Channel Reliability Model

EGAL-25 introduces:

None

Communication Reliability Levels

to govern:

None

notification routing.

Reliability Level — LOW

Definition:

None

best-effort delivery.

Failure is:

None

acceptable.

Examples:

- welcome message
- friendly reminder
- informational update

Channels:

None

IN_APP

ZALO_PZ

Reliability Level — MEDIUM

Definition:

None

usually reliable

but non-critical.

Channels:

None

TELEGRAM

WHATSAPP

WEB_PUSH

Examples:

- clan invitation
- reminder
- activity notice

Reliability Level — HIGH

Definition:

None

important operational communication.

Failure becomes:

None

undesirable.

Primary channel:

None

EMAIL

Examples:

- password reset
 - role approval
 - account update
-

Reliability Level — VERY HIGH

Definition:

None

security-sensitive communication.

Must prioritize:

None

certainty.

Examples:

- security alert
- account compromise
- administrator recovery

Primary requirement:

None

EMAIL mandatory

Optional future:

None

SMS fallback

7.5 Zalo Personal Strategy

Strategic Position

EGAL-25 intentionally chooses:

None

Personal Zalo

instead of:

None

Zalo OA.

Reason:

Zalo remains:

None

the dominant communication habit

for many:

None

elderly Vietnamese users.

Ignoring this reality would violate:

None

Elder Support Philosophy.

Architectural Constraint

Personal Zalo lacks:

None

stable official automation APIs.

Therefore:

the system SHALL NOT rely on:

None

reverse-engineered automation

as production infrastructure.

Final Decision

Zalo Personal SHALL become:

None

Manual-Assisted Channel

Meaning:

Supported

None

QR onboarding

deep links

copy-to-send

prepared templates

admin notification queue

human conversation

Forbidden

None

bot automation

reverse-engineered API dependency

identity coupling

Elder Communication Flow

Example:

Step 1

Register:

None

Preferred channel:

Zalo

Step 2

Approval pending:

User sees:

None

You selected Zalo

as your preferred communication channel.

Would you like to connect

with Clan Admin via Zalo?

Step 3

Optional:

None

Scan QR

or:

None

Open Zalo

Step 4

Admin manually communicates.

Supported by:

None

prepared templates

manual queue

Doctrine Statement

None

Zalo supports relationships.

myClan owns workflow.

7.6 Telegram Strategy

Strategic Position

Telegram SHALL be treated as:

None

automation-friendly channel.

Reason:

Telegram provides:

- stable APIs
- free bot ecosystem
- easy integration
- global availability

Recommended Use Cases

Preferred for:

None

approval notice

password reset

reminders

editor workflow

admin notification

Recommended User Segments

Best suited for:

None

young Vietnamese users

technical users

admins

editors

Doctrine Statement

None

Telegram improves automation

without heavy dependency.

7.7 WhatsApp Strategy

Strategic Position

WhatsApp SHALL remain:

None

secondary priority

during:

None

EGAL-25.x.x

Reason:

- Meta dependency
- operational cost concerns
- business API restrictions

Intended Audience

Best suited for:

None

international relatives

overseas clan members

Recommendation

WhatsApp support SHALL remain:

None

best-effort convenience layer.

NOT:

None

core infrastructure.

Doctrine Statement

None

International support matters.

But dependency must stay controlled.

7.8 Web Push Strategy

Strategic Position

Web Push SHALL be:

None

optional convenience feature.

NOT:

None

mandatory communication layer.

Strengths

Advantages:

- free
- real-time
- browser native
- no extra app needed

Limitations

Weaknesses:

None

permission dependent

device dependent

browser dependent

subscription fragile

Especially:

None

elderly users

may accidentally:

None

deny permission.

Recommended Usage

Best for:

None

young users

frequent users

real-time reminders

Doctrine Statement

None

Web Push improves convenience.

Never depend on it.

7.9 Multi-Layer Communication Model

EGAL-25 introduces:

None

Three-Layer Communication Model

Layer 1 — Mandatory Channel

Required delivery.

Example:

None

EMAIL

Especially:

None

USER

EDITOR

ADMIN

Layer 2 — Preferred Channel

User-selected preference.

Examples:

None

ZALO

TELEGRAM

WHATSAPP

Layer 3 — Opportunistic Channel

Convenience-only delivery.

Examples:

None

IN_APP

WEB_PUSH

Example

Elder Viewer

None

Preferred:

ZALO

Convenience:

IN_APP

Young Editor

None

Mandatory:

EMAIL

Preferred:

TELEGRAM

Convenience:

WEB_PUSH

IN_APP

Admin

None

Mandatory:

EMAIL

Preferred:

TELEGRAM

Optional:

ZALO

7.10 Channel Strategy Doctrine

Summary

EGAL-25 freezes:

C1

None

Channels are adapters.

C2

None

Business logic never directly sends messages.

C3

None

Every notification begins in-app.

C4

None

Email remains official communication.

C5

None

Zalo Personal = Manual-Assisted.

C6

None

Telegram = Automation-Friendly.

C7

None

WhatsApp = International Convenience.

C8

None

Web Push = Optional Convenience.

C9

None

Mandatory

+

Preferred

+

Opportunistic

communication model SHALL govern routing.

PART VII — FINAL CHANNEL PHILOSOPHY

Core philosophy:

None

Communication channels come and go.

myClan communication must survive.

Therefore:

None

platform dependency

must always remain optional.

PART VIII — DEFERRED

COMMUNICATION BINDING

DOCTRINE

8.1 Purpose of This Doctrine

This doctrine defines:

None

when

how

and why

communication channels become associated with users in:

None

EGAL-25.x.x

Historically, many systems force users to:

None

connect communication accounts

during:

None

registration

Examples:

None

Connect Telegram

Verify WhatsApp

Bind social account

Complete channel verification

This creates:

None

high onboarding friction.

For:

None

elderly users

this becomes especially problematic.

EGAL-25 therefore introduces:

None

Deferred Communication Binding

as a foundational doctrine.

8.2 Core Principle

Communication connection SHALL happen:

None

later

NOT:

None

during initial onboarding.

Registration SHALL prioritize:

None

simplicity

confidence

low anxiety

before:

None

channel sophistication.

Meaning:

users first become:

None

successful users

before becoming:

None

fully connected users.

8.3 Problem with Immediate Binding

Traditional onboarding often assumes:

None

more setup

=

better experience.

Reality shows:

None

more setup

=

more abandonment.

Especially for:

None

elderly users.

Example of Bad Onboarding

User registers.

Immediately required:

None

verify email

connect Telegram

connect Zalo

allow notification

scan QR

confirm permissions

This creates:

None

stress

confusion

fatigue

drop-off

Result:

None

registration failure.

This directly violates:

None

EGAL-24 Elder Support Doctrine.

EGAL Principle

The system SHALL prioritize:

None

successful onboarding

over:

None

complete communication setup.

Core philosophy:

None

Simple first.

Optional later.

8.4 Registration Doctrine

During:

None

Register / Onboarding

the system SHALL ask only:

None

communication preference

NOT:

None

communication verification.

Allowed During Registration

Examples:

Preferred Channel

User selects:

None

Zalo

Telegram

WhatsApp

Email

Preferred Style

Example:

None

Receive reminders

Receive approval notices

Receive support messages

Accessibility Preferences

Example:

None

Voice support

ELDER_MODE

Large UI

Forbidden During Registration

The system SHALL NOT require:

None

Telegram bot setup

mandatory QR scan

Zalo friendship

WhatsApp verification

push permission

during onboarding.

Reason:

None

too much complexity

too early.

Doctrine Statement

None

Preference first.

Connection later.

8.5 Deferred Connection Model

After registration:

communication binding becomes:

None

progressive.

The system SHALL allow:

None

gradual communication setup.

Meaning:

users may connect channels:

None

when useful

instead of:

None

when forced.

Communication Lifecycle

Stage 1 — Registration

User declares:

None

preferred channel.

Example:

None

Zalo

No binding required.

Stage 2 — First Successful Login

System gently reminds:

None

You selected Zalo

as your preferred communication channel.

Would you like to connect

with Clan Admin via Zalo?

User MAY:

None

skip.

Stage 3 — Meaningful Event

Example:

None

approval pending

password recovery

support request

System MAY suggest:

None

connecting preferred channel.

Stage 4 — Natural Adoption

User connects voluntarily.

This creates:

None

higher success rate

lower anxiety

better trust.

8.6 Zalo Deferred Binding Flow

Because:

None

Personal Zalo

is central to:

None

elder Vietnamese users

EGAL-25 defines a dedicated flow.

Registration

User selects:

None

Preferred Channel:

Zalo

No action required.

Later Reminder

User sees:

None

You selected Zalo

as your preferred communication channel.

To receive support through Zalo,

you may connect with Clan Admin.

Options:

None

Open Zalo

Scan QR

Maybe Later

Connection

If user chooses:

None

Open Zalo

System provides:

Deep Link

None

Open Zalo conversation

QR Guidance

None

Scan to connect

Prepared Message

Example:

None

Hello,

I am connecting myClan account

for family communication support.

Important Rule

Failure to connect SHALL NEVER:

None

`block usage.`

Reason:

Communication remains:

None

`optional support layer.`

NOT:

None

`identity requirement.`

8.7 Telegram Deferred Binding Flow

Telegram is:

None

`automation-friendly.`

Still:

binding SHALL remain:

None

`optional.`

Example:

Later prompt:

None

Would you like
to receive notifications
through Telegram?

User MAY:

None

connect bot

or:

None

skip.

Benefits

Avoids:

None

technical overwhelm.

8.8 Web Push Deferred Permission

Push permission SHALL NEVER be requested:

None

`immediately after registration.`

Reason:

Users often:

None

`deny permission`

when context is unclear.

Instead:

system SHOULD request push:

None

`after trust is established.`

Example:

None

`Would you like to receive`

`real-time clan notifications?`

after:

None

multiple successful logins.

8.9 Communication Binding States

EGAL-25 defines:

None

binding lifecycle states.

State 1 — DECLARED

Meaning:

None

user preference exists

but no connection.

State 2 — SUGGESTED

Meaning:

None

system suggested setup.

State 3 — CONNECTED

Meaning:

None

channel available.

State 4 — DISABLED

Meaning:

None

user opted out

or:

None

channel no longer usable.

Example Lifecycle

None

DECLARE

↓

SUGGEST

↓

CONNECT

↓

OPTIONAL DISABLE

8.10 Business Rules

Rule DCB-1

Registration SHALL NOT fail because:

None

communication setup incomplete.

Rule DCB-2

Communication setup SHALL remain:

None

optional.

unless:

None

official email requirement

applies.

Rule DCB-3

Privileged roles:

None

USER

EDITOR

ADMIN

still require:

None

mandatory email.

Rule DCB-4

The system SHALL prefer:

None

contextual prompting

instead of:

None

forced onboarding setup.

Rule DCB-5

The system SHALL preserve:

None

low anxiety onboarding.

at all times.

8.11 Deferred Binding Doctrine

Summary

EGAL-25 freezes:

DCB-1

None

Simple first.

DCB-2

None

Preference first.

Connection later.

DCB-3

None

No forced communication setup.

DCB-4

None

Failure to connect

never blocks usage.

DCB-5

None

Trust before permission.

PART VIII — FINAL DOCTRINE

PHILOSOPHY

Core philosophy:

None

Users join family first.

Channels come later.

The communication architecture SHALL always prioritize:

None

confidence

trust

simplicity

over:

None

technical completeness.

PART IX — NOTIFICATION ORCHESTRATION MODEL

9.1 Purpose of Notification Orchestration

The communication architecture of:

None

EGAL-25.x.x

requires:

None

centralized notification governance.

Without orchestration:

communication behavior becomes:

None

scattered

inconsistent

hardcoded

unmaintainable.

Examples of bad architecture:

None

RegisterForm → send email

PasswordReset → send Telegram

Approval → send Zalo

Admin panel → send push

This creates:

None

business logic coupling

duplicate communication logic

unpredictable behavior.

EGAL-25 therefore introduces:

None

Notification Orchestrator

as:

None

central communication infrastructure.

9.2 Core Philosophy

Business logic SHALL NOT decide:

None

how communication happens.

Business logic only decides:

None

what happened.

The communication system decides:

None

who gets notified

through which channels

with what priority

at what reliability

Core philosophy:

None

Business emits intent.

Orchestrator handles delivery.

9.3 Architectural Model

EGAL-25 introduces:

None

Event → Intent → Policy → Delivery

architecture.

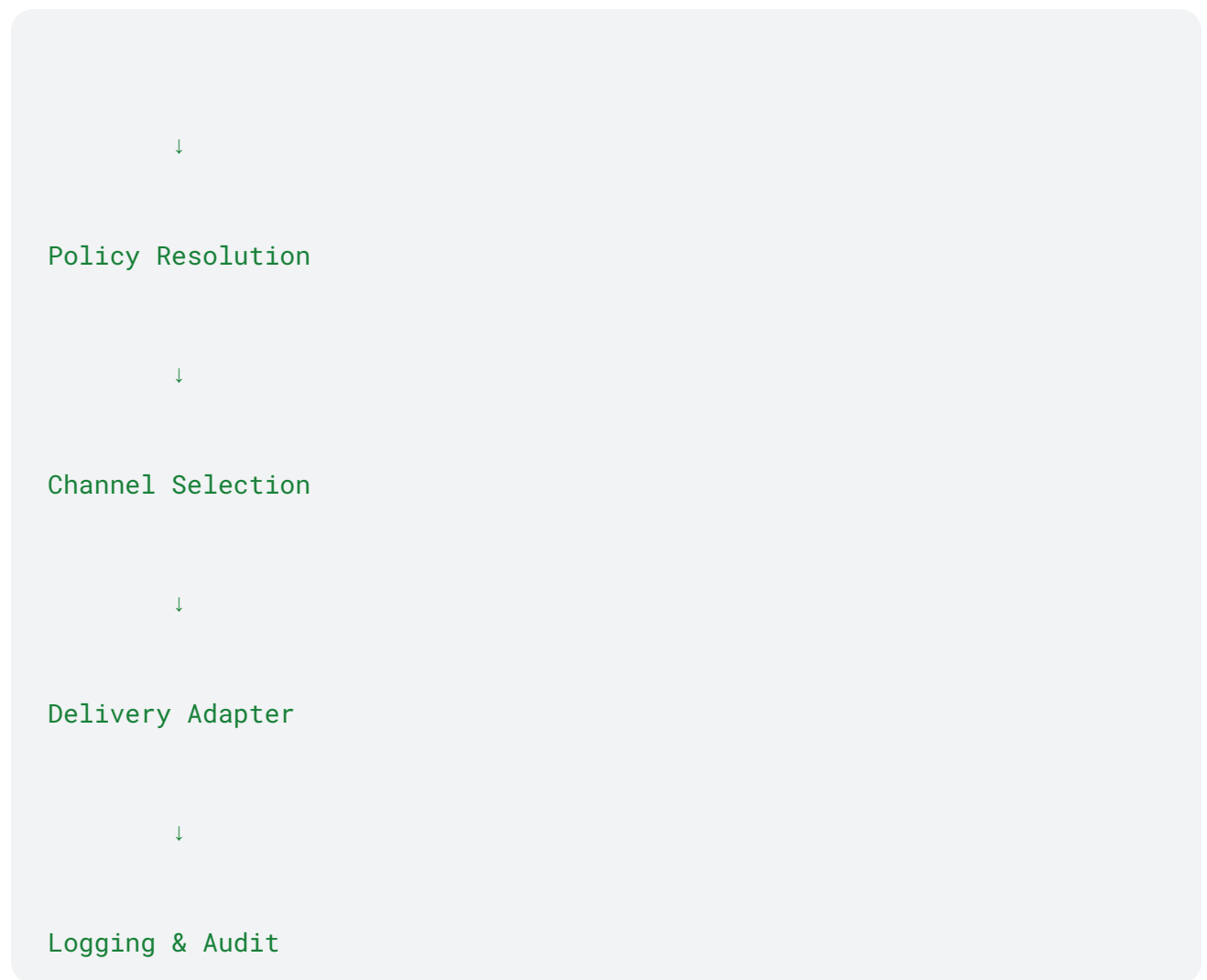
Flow

None

Business Event

↓

Notification Intent



This becomes:

None

single communication pipeline.

for:

None

all business modules.

9.4 Business Events

Business modules SHALL emit:

None

`domain events.`

NOT:

None

`communication instructions.`

Example:

Allowed

None

`USER_REGISTERED`

None

`USER_APPROVED`

None

`PASSWORD_RESET_REQUESTED`

None

PASSWORD_CHANGED

None

ROLE_CHANGED

None

SECURITY_ALERT

Forbidden

Business logic directly calling:

None

sendEmail()

sendTelegram()

sendZalo()

sendWhatsApp()

inside:

None

RegisterForm

AuthService

ApprovalWorkflow

Doctrine Statement

None

Business modules

never own communication.

9.5 Notification Intent Layer

The orchestrator converts:

None

business event

into:

None

notification intent.

Intent answers:

None

what communication

should happen.

Example:

Business event:

None

USER_APPROVED

Intent:

None

Notify approved user.

Metadata:

None

priority

severity

audience

recommended reliability

Example Intent Structure

TypeScript

```
{  
  
  event: "USER_APPROVED",  
  
  targetUserId: "...",  
  
  notificationType: "APPROVAL",  
  
  severity: "INFO",  
  
  recommendedReliability: "LOW"  
}
```

Benefits

Intent abstraction prevents:

None

`business-channel coupling.`

It enables:

None

`future extensibility.`

Example:

Today:

None

`EMAIL + ZALO`

Tomorrow:

None

`EMAIL + TELEGRAM + PUSH`

without changing:

None

`business code.`

9.6 Notification Policy Engine

The:

None

Policy Engine

determines:

None

who receives

which channel

under what conditions.

It evaluates:

A. User Role

Example:

None

VIEWER

USER

EDITOR

ADMIN

B. Accessibility Profile

Example:

None

ELDER_MODE

VOICE_SUPPORT

C. User Segment

Example:

None

elder Vietnamese

young Vietnamese

international user

D. Communication Preference

Example:

None

ZALO

TELEGRAM

WHATSAPP

EMAIL

E. Reliability Requirement

Example:

None

LOW

MEDIUM

HIGH

VERY_HIGH

Example Decision

Event:

None

PASSWORD_RESET

User:

None

EDITOR

Decision:

None

EMAIL mandatory

+

optional Telegram

+

IN_APP

Doctrine Statement

None

Communication policy

belongs to orchestrator.

9.7 Channel Selection Model

The orchestrator SHALL support:

None

multi-layer routing.

Layer 1 — Mandatory Channel

Required delivery.

Examples:

None

EMAIL

Especially:

None

USER

EDITOR

ADMIN

Layer 2 — Preferred Channel

User-selected.

Examples:

None

ZALO

TELEGRAM

WHATSAPP

Layer 3 — Opportunistic Channel

Convenience layer.

Examples:

None

WEB_PUSH

IN_APP

Example

Elder Viewer

Decision:

None

IN_APP

+

ZALO

Young Editor

Decision:

None

EMAIL

+

TELEGRAM

+

WEB_PUSH

Admin

Decision:

None

EMAIL mandatory

+

TELEGRAM

+

IN_APP

Doctrine Statement

None

One event.

Many channels.

One policy.

9.8 Delivery Adapter Layer

Each communication channel SHALL exist as:

None

isolated adapter.

Examples:

None

EmailAdapter

TelegramAdapter

WhatsAppAdapter

ZaloManualAdapter

WebPushAdapter

InAppAdapter

Adapters SHALL:

None

deliver only.

Adapters SHALL NOT:

None

contain business logic.

Adapter Responsibility

Allowed:

None

`send`

`retry`

`mark success`

`mark failure`

`delivery logging`

Forbidden:

None

`approval logic`

`permission logic`

`workflow branching`

Doctrine Statement

None

`Adapters deliver.`

`Orchestrator decides.`

9.9 Notification Logging & Audit

Every notification SHALL be:

None

logged.

Regardless of delivery success.

The system SHALL record:

None

recipient

event

channel

status

timestamp

failure reason

Example Statuses

None

PENDING

SENT

FAILED

MANUAL_REQUIRED

READ

Example

Zalo Personal:

None

MANUAL_REQUIRED

Telegram:

None

SENT

Email failure:

None

FAILED

Benefits

Auditability

Admins understand:

None

what happened.

Retryability

Failed notifications can be:

None

resent.

Debuggability

Support becomes:

None

simpler.

Doctrine Statement

None

`Communication must be observable.`

9.10 Notification Severity Model

EGAL-25 introduces:

None

`notification severity.`

INFO

Low priority.

Examples:

None

`welcome`

`friendly reminder`

`activity notice`

Reliability:

None

LOW

WARNING

Attention needed.

Examples:

None

approval pending

profile incomplete

Reliability:

None

MEDIUM

IMPORTANT

Operational significance.

Examples:

None

password reset

role change

Reliability:

None

HIGH

CRITICAL

Security-sensitive.

Examples:

None

security alert

admin recovery

account compromise

Reliability:

None

VERY_HIGH

Doctrine Statement

None

Severity governs reliability.

9.11 Notification Queue Model

EGAL-25 SHALL support:

None

asynchronous notification queue.

Meaning:

business logic SHALL NOT wait for:

None

communication delivery.

Example:

Wrong:

None

Register

→ wait for Telegram

→ wait for email

→ continue

Correct:

None

Register

→ create notification intent

→ enqueue

→ continue

Worker:

None

handles delivery later.

Benefits:

- faster UX
- retry support
- resilience
- failure isolation

Doctrine Statement

None

Communication should never block workflow.

9.12 Notification Orchestrator

Doctrine Summary

EGAL-25 freezes:

N1

None

Business emits events.

N2

None

Orchestrator owns communication.

N3

None

Channels are adapters.

N4

None

Every notification is logged.

N5

None

Severity determines reliability.

N6

None

Communication never blocks business flow.

N7

None

One event.

Many channels.

PART IX — FINAL ORCHESTRATION

PHILOSOPHY

Core philosophy:

None

Business should stay clean.

Communication should stay centralized.

EGAL-25 therefore establishes:

None

Notification Orchestrator

as:

None

foundational infrastructure

for:

None

all future communication behavior.

PART X — COMMUNICATION

RELIABILITY MODEL

10.1 Purpose of Reliability Model

The communication architecture of:

None

`EGAL-25.x.x`

must recognize a difficult reality:

None

`not all messages`

`are equally important.`

Examples:

These messages are clearly different:

None

`Welcome to myClan`

vs.

None

`Password reset required`

vs.

None

Admin account security alert

Treating all communication equally creates:

None

over-engineering

notification fatigue

poor prioritization

unnecessary cost

EGAL-25 therefore introduces:

None

Communication Reliability Model

to determine:

None

how reliable

communication must be

for each business event.

10.2 Foundational Principle

Communication SHALL follow:

None

importance-aware delivery.

Meaning:

Higher consequence events require:

None

higher communication certainty.

Lower consequence events may tolerate:

None

best-effort delivery.

Core philosophy:

None

Not every message

deserves maximum reliability.

10.3 Reliability Levels

EGAL-25 introduces:

None

four reliability levels.

LEVEL 1 — LOW

Definition

Best-effort communication.

Failure is:

None

acceptable.

Meaning:

if communication fails:

None

business continuity survives.

Typical Events

Examples:

None

welcome message

friendly reminder

informational update

suggestion

optional onboarding tip

Recommended Channels

Examples:

None

IN_APP

ZALO_PZ

WEB_PUSH

Example

Event

None

WELCOME_MESSAGE

Elder Viewer

Delivery:

None

IN_APP

+

optional ZALO

If failed:

None

acceptable.

Doctrine Statement

None

LOW reliability

never blocks workflow.

LEVEL 2 — MEDIUM

Definition

Useful communication.

Failure is:

None

undesirable

but tolerable.

Typical Events

Examples:

None

approval reminder

clan invitation

activity notice

pending review reminder

Recommended Channels

Examples:

None

TELEGRAM

WHATSAPP

IN_APP

EMAIL optional

Example

Event

None

CLAN_APPROVAL_PENDING

Young User

Delivery:

None

TELEGRAM

+

IN_APP

Fallback:

None

EMAIL

Doctrine Statement

None

MEDIUM reliability

prefers responsiveness.

LEVEL 3 — HIGH

Definition

Operationally important communication.

Failure becomes:

None

significant risk.

Communication certainty becomes:

None

important.

Typical Events

Examples:

None

password reset

role change

membership approval

important workflow change

Mandatory Channel

Required:

None

EMAIL

Optional:

None

TELEGRAM

ZALO

WEB_PUSH

IN_APP

depending on:

None

segment

preference

role.

Example

Event

None

PASSWORD_RESET_REQUESTED

USER

Delivery:

None

EMAIL mandatory

+

optional TELEGRAM

+

IN_APP

Elder Viewer

Delivery:

None

EMAIL if available

+

manual assistance

+

IN_APP

Doctrine Statement

None

HIGH reliability

requires official delivery.

LEVEL 4 — VERY HIGH

Definition

Security-sensitive communication.

Failure becomes:

None

unacceptable.

The system must maximize:

None

delivery certainty.

Typical Events

Examples:

None

security alert

account compromise

admin password recovery

critical permission change

suspicious login

Mandatory Channels

Required:

None

EMAIL

Strongly recommended:

None

TELEGRAM

Future optional:

None

SMS fallback

Example

Event

None

SUSPICIOUS_LOGIN

ADMIN

Delivery:

None

EMAIL mandatory

+

TELEGRAM

+

IN_APP

Future:

None

SMS escalation

Doctrine Statement

None

VERY HIGH reliability

prioritizes certainty

over convenience.



10.4 Reliability-to-Channel Matrix

R	Me	Channels
e	an	
l	in	
i	g	
a		
b		
i		
l		
i		
t		
y		
L	Be	In-App, Zalo, Push
O	st	
W	eff	
	ort	

M	Us	Telegram, WhatsApp, In-App
E	efu	
D	I	
I		
U		
M		
H	Im	Email mandatory + optional channels
I	po	
G	rta	
H	nt	
V	Se	Email mandatory + Telegram (+ future
E	cur	SMS)
R	ity-	
Y	crit	
H	ica	
I	I	
G		
H		

10.5 Event Classification Model

Each notification event SHALL define:

None

`minimum reliability requirement.`

Example mapping:

Event	Relationship
Welcome message	Low
Friendly onboarding reminder	Low
Approval reminder	Medium
Clan invitation	Medium

	D I U M
Password reset	H I G H
Membership approval	H I G H
Role change	H I G H
Security alert	V E R Y H I G H
Suspicious login	V E

R
Y
H
I
G
H

Admin recovery

V
E
R
Y
H
I
G
H

Doctrine Statement

None

Severity determines reliability.

NOT:

None

user preference alone.

10.6 Reliability Resolution Logic

Communication routing SHALL follow:

None

Reliability

+

Role

+

Preference

+

Accessibility

Decision order:

Step 1

Determine:

None

event severity.

Step 2

Resolve:

None

minimum reliability.

Step 3

Check:

None

mandatory channels.

Step 4

Add:

None

preferred channels.

Step 5

Add:

None

opportunistic channels.

Example

Event

None

ROLE_CHANGED

Severity:

None

HIGH

User:

None

EDITOR

Preference:

None

Telegram

Decision:

None

EMAIL mandatory

+

TELEGRAM

+

IN_APP

Doctrine Statement

None

Preference matters.

Reliability decides.

10.7 Reliability Escalation Model

EGAL-25 introduces:

None

progressive escalation.

Meaning:

if delivery fails:

system MAY escalate.

Example

Attempt 1

None

Telegram

Failed.

Attempt 2

None

EMAIL

Attempt 3

None

IN_APP

Future

None

SMS escalation

for:

None

VERY_HIGH

events.

Doctrine Statement

None

Communication should retry intelligently.

10.8 Notification Fatigue Prevention

EGAL-25 SHALL prevent:

None

notification overload.

The system SHALL avoid:

None

same message

many channels

too frequently.

Example:

Wrong:

None

Email

Telegram

Push

In-App

Zalo

all at once

for:

None

low-value reminder.

Correct:

None

single preferred channel.

Rule

Lower reliability:

None

fewer channels.

Higher reliability:

None

more redundancy.

Doctrine Statement

None

Avoid spam.

Preserve trust.

10.9 Reliability Governance Rules

Rule CRM-1

Communication SHALL match:

None

event importance.

Rule CRM-2

Critical events SHALL prioritize:

None

certainty.

Rule CRM-3

Convenience channels SHALL NEVER replace:

None

official channels.

Rule CRM-4

User preference SHALL influence routing.

But SHALL NOT override:

None

minimum reliability.

Rule CRM-5

Reliability SHALL remain:

None

observable

auditable

configurable.

10.10 Communication Reliability

Doctrine Summary

EGAL-25 freezes:

CRM-1

None

LOW = best effort.

CRM-2

None

MEDIUM = responsive.

CRM-3

None

HIGH = official delivery.

CRM-4

None

VERY_HIGH = certainty first.

CRM-5

None

Reliability governs routing.

CRM-6

None

Preference influences.

Reliability decides.

CRM-7

None

Avoid notification fatigue.

PART X — FINAL RELIABILITY PHILOSOPHY

Core philosophy:

None

Important communication

deserves stronger guarantees.

EGAL-25 therefore ensures:

None

communication effort

matches communication importance.

PART XI — REJECTED ALTERNATIVES & ARCHITECTURAL TRADEOFFS

11.1 Purpose of This Section

This section documents:

None

architectural decisions

that were intentionally rejected.

The purpose is:

None

future clarity.

Without documenting rejected alternatives:

future implementation teams may:

None

reintroduce old mistakes

repeat abandoned approaches

misunderstand architecture rationale.

EGAL-25 therefore formally records:

None

why certain options

were NOT selected.

This section is intentionally:

None

decision-oriented

rather than:

None

technology-oriented.

11.2 Rejected Alternative A

Social Platforms as Identity Providers

Rejected Proposal

Use:

None

Zalo login

Telegram login

WhatsApp login

as:

None

authentication identity.

Equivalent model:

None

social account

=

myClan account

Why It Was Considered

Advantages initially appeared attractive:

- faster onboarding
- less password management
- easier login experience
- familiar user habit

Especially:

None

elder users

may appear more comfortable with:

None

Zalo

than:

None

email/password.

Why It Was Rejected

Problem 1 — Vendor Lock-in

Identity becomes dependent on:

None

third-party platform survival.

Example:

If:

None

Zalo policy changes

Telegram account lost

WhatsApp blocked

then:

None

account access breaks.

This is unacceptable for:

None

long-lived genealogy systems.

Problem 2 — Platform Dependency

myClan becomes:

None

externally controlled.

Future architectural freedom decreases.

Problem 3 — Security Fragility

Authentication assurance becomes:

None

outside system control.

Especially dangerous for:

None

EDITOR

ADMIN

roles.

Problem 4 — Migration Difficulty

Changing communication platform later becomes:

None

identity migration problem.

Instead of:

None

simple preference update.

Final Decision

Rejected.

Replaced by:

None

Identity $\neq$ Communication

Doctrine Statement

None

Social platforms may change.

Identity must survive.

11.3 Rejected Alternative B

Zalo OA Dependency

Rejected Proposal

Use:

None

Zalo Official Account (OA)

as:

None

primary communication infrastructure.

Why It Was Considered

Potential benefits:

- official API
- automation support
- webhooks
- scalable messaging

Why It Was Rejected

Problem 1 — Strategic Independence

EGAL intentionally prefers:

None

free-first architecture.

OA introduces:

None

platform dependency.

Problem 2 — Governance Risk

myClan communication becomes dependent on:

None

Zalo platform policies.

Future changes may affect:

None

message quotas

business restrictions

approval requirements

Problem 3 — Misalignment with Elder Reality

Real-world behavior shows:

many elderly users already use:

None

Personal Zalo.

They naturally communicate:

None

human ↔ human

rather than:

None

human ↔ organization account.

Problem 4 — Architectural Philosophy Conflict

EGAL prioritizes:

None

human-centered assistance

rather than:

None

forced automation.

Final Decision

Rejected.

Replaced by:

None

Personal Zalo

=

Manual-Assisted Channel.

Doctrine Statement

None

Human relationship first.

Automation second.

11.4 Rejected Alternative C

Unofficial Personal Zalo Automation

Rejected Proposal

Use:

None

reverse-engineered

Personal Zalo APIs

Examples:

None

web automation

browser emulation

session hijacking

unofficial libraries

Why It Was Considered

Potential benefits:

None

free automation

no OA dependency

full Zalo integration

Why It Was Rejected

Problem 1 — Production Instability

Such solutions depend on:

None

private implementation details.

Zalo updates may instantly break:

None

automation behavior.

Problem 2 — Account Ban Risk

Personal accounts risk:

None

temporary restriction

permanent banning

especially for:

None

`Clan Admin.`

This is operationally unacceptable.

Problem 3 — Security Risk

Unofficial APIs often require:

None

`session persistence`

`token extraction`

`browser automation`

This conflicts with:

None

`security-first design.`

Problem 4 — Long-Term Maintainability

The architecture becomes:

None

`fragile`

high maintenance

difficult to debug.

Final Decision

Rejected.

Replaced by:

None

Manual-Assisted Personal Zalo

with:

None

QR

deep links

copy-to-send

manual queue.

Doctrine Statement

None

Stable systems

outlive clever hacks.

11.5 Rejected Alternative D

Mandatory Communication Binding During Registration

Rejected Proposal

Require users to:

None

connect Telegram

scan Zalo QR

allow push permission

verify social account

before:

None

registration completion.

Why It Was Considered

Potential benefits:

- faster activation
- higher connection rate
- better automation

Why It Was Rejected

Problem 1 — Elder Anxiety

Too much setup causes:

None

stress

confusion

abandonment.

Especially harmful for:

None

elder users.

Problem 2 — Violates EGAL-24 Doctrine

EGAL-24 established:

None

low anxiety onboarding.

Mandatory setup conflicts with:

None

simple first.

Problem 3 — Premature Complexity

Users have not yet developed:

None

trust

context

motivation

to complete communication setup.

Final Decision

Rejected.

Replaced by:

None

Deferred Communication Binding

Doctrine Statement

None

Trust first.

Connection later.

11.6 Rejected Alternative E

Single Preferred Channel Model

Rejected Proposal

Store:

None

one preferred channel only.

Example:

None

preferredChannel = ZALO

Why It Was Considered

Advantages:

- simpler DB
 - easier implementation
 - lower complexity
-

Why It Was Rejected

Problem 1 — Reliability Weakness

Single channel creates:

None

single point of failure.

Example:

If:

None

Telegram unavailable

then:

None

communication fails.

Problem 2 — No Official Communication Separation

System cannot distinguish:

None

official

vs:

None

convenience

communication.

Problem 3 — Real User Behavior

Users naturally use:

None

multiple channels.

Example:

None

email

+

Telegram

+

in-app

Final Decision

Rejected.

Replaced by:

None

Mandatory

+

Preferred

+

Opportunistic

communication model.

Doctrine Statement

None

Communication requires redundancy.

11.7 Rejected Alternative F

Email-Only Communication

Rejected Proposal

Keep:

None

EGAL-24

email-only architecture.

Why It Was Considered

Advantages:

- simple
 - reliable
 - already implemented
-

Why It Was Rejected

Problem 1 — Elder Disconnect

Many elderly users:

None

rarely check email.

Result:

None

missed communication.

Problem 2 — Poor Engagement

The system fails to meet users:

None

where they already communicate.

Problem 3 — International Limitation

International relatives prefer:

None

WhatsApp

Telegram.

Final Decision

Rejected.

Replaced by:

None

multi-channel architecture.

Doctrine Statement

None

Users should not adapt

to communication assumptions.

The system should adapt to users.

11.8 Architectural Tradeoff Summary

EGAL-25 intentionally chooses:

None

practicality

maintainability

elder friendliness

resilience

over:

None

maximum automation

technical cleverness

platform dependency

This reflects:

None

long-term genealogy platform thinking.

11.9 Rejected Alternative Doctrine

Summary

EGAL-25 freezes:

RA-1

None

No social identity coupling.

RA-2

None

No Zalo OA dependency.

RA-3

None

No unofficial PZ automation.

RA-4

None

No forced communication setup.

RA-5

None

No single-channel dependency.

RA-6

None

No email-only future.

PART XI — FINAL ARCHITECTURAL PHILOSOPHY

Core philosophy:

None

Architecture is defined

not only by what is chosen,

but also by

what is intentionally rejected.

These rejected alternatives SHALL remain:

None

institutional architectural memory

for:

None

future EGAL evolution.

PART XII — IMPLEMENTATION

CONSEQUENCES & SYSTEM

IMPACT

12.1 Purpose of This Section

This section formalizes:

None

what EGAL-25 architecture changes imply

for:

None

backend

frontend

database

business flow

UX

operations

An Architecture Decision Record (ADR) is incomplete without understanding:

None

implementation consequences.

Every architectural decision creates:

None

technical obligations.

EGAL-25 therefore documents:

None

required system evolution

before implementation begins.

12.2 Foundational Consequence

The introduction of:

None

Unified Communication Layer (UCL)

means:

communication can no longer remain:

None

hardcoded

inside:

None

forms

pages

business services

Examples of forbidden future design:

None

RegisterForm → send email

ApproveUser → send telegram

`ResetPassword → send notification directly`

Instead:

all communication SHALL pass through:

None

`Notification Orchestrator.`

This becomes:

None

`foundational infrastructure.`

12.3 Architectural Layer Impact

EGAL-25 impacts:

None

`all major system layers.`

Layer A — Database Layer

Required Change

Current schema is insufficient.

The system SHALL introduce:

None

communication-aware schema.

New Concepts Required

Communication preference

The system must know:

None

how users prefer communication.

Notification persistence

The system must remember:

None

every notification.

Delivery status

The system must know:

None

what succeeded

what failed

what requires manual action.

Required New Data Structures

At minimum:

None

UserContactPreference

Notification

NotificationDeliveryLog

Architectural Consequence

Database complexity:

None

increases moderately.

But:

system resilience increases:

None

dramatically.

Doctrine Statement

None

Communication memory

belongs in database.

Layer B — Backend Layer

Required Change

Business services SHALL stop owning:

None

communication delivery.

Instead:

services emit:

None

notification intent.

Required Modules

The backend SHALL introduce:

None

notification module

policy engine

adapter layer

delivery queue

Example structure:

None

modules/

notifications/

orchestrator/

adapters/

policy/

queue/

New Responsibility Model

Before EGAL-25

Business flow:

None

`business logic`

+

`communication logic`

inside same service.

After EGAL-25

Business flow:

None

`business logic`

ONLY.

Communication becomes:

None

`infrastructure concern.`

Example

Wrong:

None

```
approveUser()
```

```
sendEmail()
```

Correct:

None

```
approveUser()
```

```
emit(USER_APPROVED)
```

Orchestrator decides:

None

```
how to notify.
```

Doctrine Statement

None

```
Business logic
```

```
must stay communication-agnostic.
```

Layer C — Frontend Layer

Required Change

The frontend SHALL become:

None

`communication-aware.`

without becoming:

None

`communication-dependent.`

Meaning:

UI must support:

None

`channel preference`

`notification center`

`manual-assisted flows`

but SHALL NOT contain:

None

`communication business logic.`

New Frontend Components

Likely required:

None

`NotificationCenter`

`NotificationBell`

`ContactPreferenceSelector`

`ZaloQrGuide`

`AdminManualQueue`

UX Consequence

Users gain:

None

`more communication flexibility.`

But:

UX complexity risks increasing.

Thus:

EGAL-24 principles remain mandatory:

None

low anxiety

simple first

controlled complexity.

Doctrine Statement

None

Communication power

must not increase UI anxiety.

Layer D — Security Layer

Required Change

Communication SHALL remain:

None

non-authoritative.

Meaning:

channels MUST NOT become:

None

identity dependencies.

Forbidden:

None

login via Zalo

Telegram-auth requirement

mandatory WhatsApp verification

Email Security Exception

For:

None

USER

EDITOR

ADMIN

Email remains:

None

official channel.

Because:

security workflows require:

None

reliable recovery.

Consequence

Authentication architecture:

None

remains stable.

No major rewrite required.

Doctrine Statement

None

Communication evolves.

Authentication remains stable.

Layer E — Operational Layer

Required Change

Admins SHALL gain:

None

communication operations tooling.

Especially for:

None

Zalo Manual-Assisted Flow.

Required Admin Features

Examples:

Notification queue

Admin sees:

None

messages waiting to be sent.

Manual Zalo support

Admin gets:

None

copy-ready messages

deep links

user guidance

Delivery visibility

Admin sees:

None

SENT

FAILED

MANUAL_REQUIRED

Consequence

Admin workload:

None

slightly increases.

But:

communication quality:

None

improves dramatically.

Especially for:

None

elder users.

Doctrine Statement

None

Human assistance

is part of system design.

12.4 Business Flow Impact

Several business flows SHALL evolve.

Register / Onboarding

Before:

None

`register`

→ `email only`

After:

None

`register`

→ `communication preference`

→ `optional later binding`

Approval Flow

Before:

None

`approve`

→ `email only`

After:

None

`approve`

→ notification intent

→ policy engine

→ multi-channel delivery

Password Reset

Before:

None

email-centric.

After:

None

email mandatory

+

optional convenience channels

depending on:

None

role

segment

reliability.

Login Reminder

New capability:

System MAY remind:

None

Would you like

to connect your preferred channel?

without blocking usage.

Doctrine Statement

None

Communication improves workflow.

Never interrupts workflow.

12.5 Technical Debt Consequence

EGAL-25 intentionally creates:

None

`moderate complexity increase.`

Tradeoff accepted.

Reason:

the architecture gains:

None

`maintainability`

`scalability`

`flexibility`

`segment-awareness`

without:

None

`platform lock-in.`

Debt Avoided

EGAL-25 intentionally avoids:

None

social identity coupling

unofficial Zalo automation

hardcoded notification logic

email-only future

These avoided debts are considered:

None

high-risk technical debt.

12.6 Migration Impact

EGAL-25 SHALL remain:

None

incrementally adoptable.

No:

None

big bang rewrite.

required.

Migration Strategy

Phase 1

Introduce:

None

notification persistence.

No UI changes.

Phase 2

Introduce:

None

communication preference.

during onboarding.

Phase 3

Introduce:

None

notification orchestrator.

Phase 4

Introduce:

None

Telegram automation.

Phase 5

Introduce:

None

Zalo manual-assisted queue.

Phase 6

Optional:

None

Web Push.

Doctrine Statement

None

Evolution

must remain controlled.

12.7 Implementation Consequence

Summary

EGAL-25 requires:

IC-1

None

communication infrastructure.

IC-2

None

notification persistence.

IC-3

None

frontend communication awareness.

IC-4

None

admin communication tooling.

IC-5

None

multi-channel routing.

IC-6

None

incremental migration.

IC-7

None

strict separation:

business

≠

communication.

PART XII — FINAL

IMPLEMENTATION PHILOSOPHY

Core philosophy:

None

Architecture decisions

must remain implementable.

EGAL-25 therefore chooses:

None

controlled evolution

over:

None

architectural rewrite.

Because:

None

stable systems

evolve carefully.

PART XIII — CONTROLLED EVOLUTION ROADMAP

13.1 Purpose of This Roadmap

The architecture of:

None

EGAL-25.x.x

Unified Communication Architecture

is intentionally designed as:

None

controlled evolution.

NOT:

None

big-bang rewrite.

Reason:

myClan already contains:

None

stable EGAL-24 functionality.

including:

- onboarding
- elder support doctrine
- accessibility guidance
- authentication workflow
- voice assistance
- low-anxiety UX

These production-safe components SHALL remain:

None

protected.

Therefore:

EGAL-25 evolution must obey:

None

Q1

Do not break working business flow.

and:

None

Q2

Preserve UX unless intentionally modified.

The roadmap below formalizes:

None

incremental adoption.

13.2 Evolution Philosophy

EGAL-25 SHALL evolve through:

None

infrastructure-first migration.

Meaning:

the system SHALL first introduce:

None

backend capability

before:

None

user-facing complexity.

This preserves:

None

elder confidence

stable onboarding

low operational risk.

Core philosophy:

None

Invisible foundation first.

Visible feature later.

13.3 EGAL-25.0

Communication Foundation

Objective

Establish:

None

communication infrastructure.

without changing:

None

existing user behavior.

This phase SHALL remain:

None

non-disruptive.

Scope

Introduce:

None

Notification persistence

Notification schema

Notification lifecycle

Communication preference model

No automation yet.

No Telegram yet.

No Zalo workflow yet.

Backend Work

Create:

None

`Notification`

`NotificationDeliveryLog`

`UserCommunicationPreference`

Introduce:

None

`Notification Orchestrator`

in:

None

`silent mode.`

Meaning:

system still behaves:

None

email-first

but notifications become:

None

persistent.

Frontend Work

Minimal UI additions.

Add:

None

Preferred Communication Method

during onboarding.

Options:

None

Email

Zalo

Telegram

WhatsApp

No forced setup.

No QR.

No binding.

Deliverables

By end of:

None

EGAL-25.0

the system SHALL have:

notification database

communication preference

orchestration foundation

no user disruption

Risk Level

None

LOW

Doctrine Statement

None

`Build foundation quietly.`

13.4 EGAL-25.1

Telegram Automation Layer

Objective

Introduce:

None

`real automation`

through:

None

`Telegram.`

Telegram becomes:

None

`first automation-friendly channel.`

Why Telegram First

Because Telegram is:

None

free

stable

API-friendly

international

without:

None

vendor lock-in risk.

Scope

Add:

None

Telegram Bot

Flow:

None

`connect bot`

`→ save chat id`

`→ receive notification`

Use Cases

Examples:

None

`approval notification`

`password reset notice`

`security reminder`

`admin alert`

`editor workflow`

Frontend Work

Add:

None

Connect Telegram

inside:

None

Communication Preferences.

Optional only.

Deliverables

By end of:

None

EGAL-25.1

system supports:

None

real multi-channel automation.

Risk Level

None

LOW-MEDIUM

Doctrine Statement

None

Automate where stability exists.

13.5 EGAL-25.2

Notification Orchestration Activation

Objective

Turn:

None

Notification Orchestrator

from:

None

passive

into:

None

active routing engine.

Scope

Business modules SHALL stop:

None

`sending directly.`

Instead:

they emit:

None

`notification intent.`

Migration

Before:

None

`RegisterForm`

`→ send email`

After:

None

`RegisterForm`

```
→ emit(USER_REGISTERED)
```

Modules to Migrate

Examples:

None

Register

Forgot Password

Approval

Role Change

Security Alert

Deliverables

Centralized:

None

notification routing.

Risk Level

None

MEDIUM

because:

business flows are affected.

Doctrine Statement

None

Centralize communication gradually.

13.6 EGAL-25.3

Zalo Manual-Assisted Layer

Objective

Introduce:

None

elder-friendly communication.

through:

None

Personal Zalo.

without:

None

automation dependency.

Scope

Introduce:

None

QR guidance

Open Zalo

Prepared messages

Manual send queue

Example Flow

User selected:

None

Zalo

Later:

system suggests:

None

Would you like

to connect with Clan Admin?

Options:

None

Open Zalo

Scan QR

Later

Admin Tooling

Admin receives:

None

manual queue

copy-ready messages

delivery state

Deliverables

myClan becomes:

None

elder communication friendly.

Risk Level

None

LOW

Because:

communication remains:

None

optional.

Doctrine Statement

None

Support humans.

Do not automate fragility.

13.7 EGAL-25.4

Web Push Layer

Objective

Introduce:

None

real-time convenience notifications.

Scope

Support:

None

browser push

for:

None

young users

frequent users

Usage

Best for:

None

reminders

activity update

editor collaboration

NOT:

None

critical security.

Deliverables

Optional:

None

high engagement layer.

Risk Level

None

LOW

Doctrine Statement

None

Convenience

must remain optional.

13.8 EGAL-25.5+

International Expansion Layer

Objective

Support:

None

`international relatives.`

through:

None

`WhatsApp integration.`

Scope

Limited:

None

`best-effort support.`

NOT:

None

`core dependency.`

Deliverables

Stronger:

None

`international communication.`

Risk Level

None

MEDIUM

because:

None

Meta platform dependency.

Doctrine Statement

None

Expand carefully.

Stay independent.

13.9 Migration Safety Rules

EGAL-25 SHALL obey:

Rule CER-1

None

No big bang rewrite.

Rule CER-2

None

Stable flows remain protected.

Rule CER-3

None

Infrastructure before UX.

Rule CER-4

None

Optional before mandatory.

Rule CER-5

None

Low anxiety always wins.

Rule CER-6

None

Communication never breaks onboarding.

Rule CER-7

None

Prefer maintainability over cleverness.

13.10 Timeline Summary

V	Focus	Risk
e		
r		
s		
i		
o		
n		
E	Communication	LO
G	foundation	W
A		
L		

-
2
5
.
0

E
G
A
L
-

Telegram
automation

LO
W-
ME
DIU
M

2
5
.
1

E
G
A
L
-

Notification
orchestrator

ME
DIU
M

2
5
.
2

E
G
A
L
-

Zalo
manual-assisted

LO
W

2

5

.

3

E

Web push

LO

G

W

A

L

-

2

5

.

4

E

WhatsApp

ME

G

expansion

DIU

A

M

L

-

2

5

.

5

+

13.11 Final Controlled Evolution

Summary

EGAL-25 chooses:

None

controlled evolution

instead of:

None

aggressive transformation.

Reason:

the system already possesses:

None

working business flow

working elder UX

stable onboarding

which SHALL remain:

None

protected.

PART XIII — FINAL ROADMAP

PHILOSOPHY

Core philosophy:

None

Strong systems

evolve carefully.

EGAL-25 therefore commits to:

None

incremental architecture

that improves:

None

communication

without sacrificing:

None

simplicity

stability

elder confidence.

PART XIV — DATA MODEL & SCHEMA DESIGN

14.1 Purpose of This Section

This section formalizes:

None

database evolution

required for:

None

EGAL-25.x.x

Unified Communication Architecture.

The purpose is:

None

implementation safety.

Without schema standardization:

implementation risks:

None

inconsistent data model

architecture drift

hardcoded shortcuts

migration pain

This section therefore freezes:

None

`communication data model.`

for:

None

`Prisma`

`PostgreSQL`

based deployment.

14.2 Architectural Principle

Communication SHALL become:

None

`persistent infrastructure.`

Meaning:

the system must remember:

None

`preferences`

`bindings`

notifications

delivery attempts

results

instead of:

None

fire-and-forget messaging.

Core philosophy:

None

Communication must be observable.

14.3 Schema Evolution Strategy

EGAL-25 SHALL evolve:

None

incrementally.

Existing:

None

`users`

`roles`

`tenant`

`authentication`

tables SHALL remain:

None

`stable.`

New communication tables SHALL be:

None

`additive.`

NOT:

None

`destructive rewrite.`

This preserves:

None

`EGAL-24 compatibility.`

14.4 High-Level Entity Model

EGAL-25 introduces:

None

5 communication entities.

Architecture:

None

User



UserCommunicationPreference



CommunicationBinding

Notification



NotificationRecipient



NotificationDelivery

14.5 Entity #1

UserCommunicationPreference

Purpose

Stores:

None

how users prefer communication.

This entity represents:

None

declared preference

NOT:

None

confirmed binding.

Example:

User says:

None

I prefer Zalo.

Even before:

None

Zalo connection exists.

Prisma Model

None

```
model UserCommunicationPreference {  
  
  id          String    @id @default(uuid())  
  
  userId      String    @unique  
  
  user        User      @relation(fields: [userId],  
references: [id])  
  
  preferredChannel ChannelType?  
  
  secondaryChannel ChannelType?
```



```
allowReminder          Boolean @default(true)

allowApprovalNotice    Boolean @default(true)

allowSecurityNotice    Boolean @default(true)


createdAt              DateTime @default(now())

updatedAt              DateTime @updatedAt

}
```

Design Decision

Preference is intentionally:

None

`lightweight.`

Reason:

avoids:

None

elder onboarding friction.

Doctrine Statement

None

Preference first.

Binding later.

14.6 Entity #2

CommunicationBinding

Purpose

Stores:

None

actual communication connection state.

Example:

Telegram:

None

`chat_id exists`

Zalo:

None

`QR connected`

Email:

None

`verified.`

Prisma Model

None

```
model CommunicationBinding {  
  
  id          String      @id @default(uuid())  
  
  userId      String  
  
  user        User        @relation(fields: [userId], references: [id])  
}
```

channel	ChannelType	
bindingState	BindingState	@default(DECLARED)
externalId	String?	
displayName	String?	
connectedAt	DateTime?	
disabledAt	DateTime?	
metadata	Json?	
createdAt	DateTime	@default(now())
updatedAt	DateTime	@updatedAt

```
    @@unique([userId, channel])

}
```

Explanation

externalId

Examples:

Telegram:

None

`chat_id`

Zalo:

None

`manual reference id`

WhatsApp:

None

`phone`

metadata

Flexible storage:

Example:

JSON

```
{  
  
  "deepLinkUsed": true,  
  
  "lastQrScanAt": "2026-06-03"  
  
}
```

Reason:

future-proofing.

Binding Lifecycle

States:

None

DECLARED

SUGGESTED

CONNECTED

DISABLED

Doctrine Statement

None

Binding is optional.

Preference is enough.

14.7 Entity #3

Notification

Purpose

Represents:

None

canonical notification record.

Every business event SHALL first create:

None

notification record.

before:

None

external delivery.

Prisma Model

None

```
model Notification {  
  
  id          String          @id  
  @default(uuid())  
  
  eventType   NotificationEvent  
  
  title       String  
  
  message     String
```



```
    severity          NotificationSeverity

    reliability        ReliabilityLevel

    payload            Json?

    createdBySystem    Boolean
    @default(true)

    createdAt          DateTime
    @default(now())

    recipients          NotificationRecipient[]

}
```

Design Decision

Notification remains:

None

`channel-independent.`

Meaning:

no:

None

`email field`

`telegram field`

`zalo field`

inside notification.

Reason:

prevents:

None

`channel coupling.`

Doctrine Statement

None

`Notification is intent.`

Delivery is separate.

14.8 Entity #4

NotificationRecipient

Purpose

Supports:

None

multi-recipient notification.

Example:

None

Admin warning

may notify:

None

multiple admins.

Prisma Model

None

```
model NotificationRecipient {  
  
  id          String          @id  
  @default(uuid())  
  
  notificationId String  
  
  notification Notification  
  @relation(fields: [notificationId], references: [id])  
  
  userId      String  
  
  user        User  
  @relation(fields: [userId], references: [id])  
  
  readAt      DateTime?  
  
  createdAt   DateTime  
  @default(now())  
}
```

```
deliveries
```

```
NotificationDelivery[]
```

```
}
```

Reason

Separates:

None

```
message
```

from:

None

```
recipient.
```

Improves:

None

```
reuse
```

```
auditability.
```

Doctrine Statement

None

One notification.

Many recipients.

14.9 Entity #5

NotificationDelivery

Purpose

Tracks:

None

delivery execution.

per:

None

channel.

Example:

One notification:

None

EMAIL success

Telegram fail

Zalo manual required

Prisma Model

None

```
model NotificationDelivery {  
  
  id          String          @id  
  @default(uuid())  
  
  recipientId String  
  
  recipient    NotificationRecipient  
  @relation(fields: [recipientId], references: [id])  
  
  channel      ChannelType
```

status DeliveryStatus

sentAt DateTime?

deliveredAt DateTime?

readAt DateTime?

retryCount Int @default(0)

failureReason String?

metadata Json?

createdAt DateTime
@default(now())


```
}
```

Example

Telegram:

None

SENT

Email:

None

FAILED

Zalo:

None

MANUAL_REQUIRED

Doctrine Statement

None

Delivery must be observable.

14.10 Enum Design

ChannelType

None

```
enum ChannelType {  
  
    EMAIL  
  
    ZALO_PZ  
  
    TELEGRAM  
  
    WHATSAPP  
  
    WEB_PUSH  
  
    IN_APP  
  
}
```

ReliabilityLevel

None

```
enum ReliabilityLevel {
```

```
LOW

MEDIUM

HIGH

VERY_HIGH

}
```

NotificationSeverity

None

```
enum NotificationSeverity {

    INFO

    WARNING

    IMPORTANT

    CRITICAL

}
```

DeliveryStatus

None

```
enum DeliveryStatus {  
  
    PENDING  
  
    SENT  
  
    FAILED  
  
    MANUAL_REQUIRED  
  
    READ  
  
}
```

BindingState

None

```
enum BindingState {  
  
    DECLARED  
  
    SUGGESTED  
  
    CONNECTED  
  
}
```

```
DISABLED
```

```
}
```

NotificationEvent

Temporary starter catalog:

None

```
enum NotificationEvent {  
  
    USER_REGISTERED  
  
    USER_APPROVED  
  
    PASSWORD_RESET_REQUESTED  
  
    PASSWORD_CHANGED  
  
    ROLE_CHANGED  
  
    SECURITY_ALERT  
  
    SUSPICIOUS_LOGIN  
  
}
```

Will expand in:

None

PART XV.

14.11 Relationship Diagram

Architecture:

None

User

↓ 1:1

UserCommunicationPreference

User

↓ 1:N

CommunicationBinding

Notification

↓ 1:N

NotificationRecipient

NotificationRecipient

↓ 1:N

NotificationDelivery

14.12 Migration Strategy

Phase 1

Create:

None

communication tables.

No business logic changes.

Phase 2

Silent notification persistence.

Phase 3

Policy engine integration.

Phase 4

Delivery adapters.

Doctrine Statement

None

Schema first.

Behavior later.

14.13 Data Model Doctrine Summary

EGAL-25 freezes:

DM-1

None

Preference ≠ Binding.

DM-2

None

Notification ≠ Delivery.

DM-3

None

One notification.

Many recipients.

Many deliveries.

DM-4

None

Delivery must be observable.

DM-5

None

Schema evolves additively.

PART XIV — FINAL DATA MODEL

PHILOSOPHY

Core philosophy:

None

Communication systems

must remember.

Therefore:

EGAL-25 chooses:

None

persistent communication infrastructure

instead of:

None

fire-and-forget messaging.

PART XV — NOTIFICATION EVENT

CATALOG & GOVERNANCE

15.1 Purpose of This Section

This section freezes:

None

notification governance.

for:

None

EGAL-25.x.x

Unified Communication Architecture.

The purpose is to standardize:

None

what events exist

how important they are

who receives them

how reliable delivery must be

Without a formal event catalog:

implementation risks:

None

`inconsistent naming`

`duplicated logic`

`severity mismatch`

`routing confusion`

This section therefore establishes:

None

`single source of truth`

for:

None

`notification behavior.`

15.2 Foundational Principle

Business modules SHALL emit:

None

`standardized business events.`

NOT:

None

```
channel-specific logic.
```

Example:

Correct:

None

```
USER_APPROVED
```

Wrong:

None

```
sendApprovalEmail()
```

Core philosophy:

None

```
Events describe reality.
```

```
Orchestrator decides communication.
```

15.3 Event Taxonomy

EGAL-25 classifies notification events into:

None

6 event families.

Category A — Onboarding Events

Related to:

None

registration

approval

activation.

Category B — Authentication Events

Related to:

None

login

password

identity recovery.

Category C — Clan Workflow Events

Related to:

None

clan participation

approval

collaboration.

Category D — Administrative Events

Related to:

None

role management

moderation

governance.

Category E — Security Events

Related to:

None

account safety

suspicious activity.

Category F — Engagement Events

Related to:

None

reminders

soft communication

re-engagement.

15.4 Event Governance Model

Each event SHALL define:

None

severity

reliability

mandatory channels

preferred channels

fallback strategy

target audience

Structure:

None

Event



Severity



Reliability



Routing Policy



Delivery

15.5 CATEGORY A — ONBOARDING EVENTS

EVENT

None

USER_REGISTERED

Purpose

User successfully submitted registration.

Severity

None

INFO

Reliability

None

LOW

Target

None

registered user

admin

Recommended Channels

User:

None

IN_APP

Admin:

None

EMAIL

+

TELEGRAM

Example Message

None

Your registration has been received.

EVENT

None

USER_APPROVAL_PENDING

Purpose

Registration awaits review.

Severity

None

WARNING

Reliability

None

MEDIUM

Target

None

user

Recommended Channels

None

IN_APP

+

Preferred Channel

EVENT

None

USER_APPROVED

Purpose

User approved.

Severity

None

IMPORTANT

Reliability

None

HIGH

Mandatory Channel

None

EMAIL (if exists)

Preferred Channel

None

ZALO

TELEGRAM

Example

None

Your membership has been approved.

EVENT

None

USER_REJECTED

Severity

None

IMPORTANT

Reliability

None

HIGH

Delivery

None

EMAIL mandatory

+

IN_APP

15.6 CATEGORY B — AUTHENTICATION EVENTS

EVENT

None

PASSWORD_RESET_REQUESTED

Purpose

User requested reset.

Severity

None

IMPORTANT

Reliability

None

HIGH

Mandatory Channel

None

EMAIL

Optional

None

TELEGRAM

IN_APP

Rule

Must NEVER rely only on:

None

ZALO

WEB_PUSH

EVENT

None

PASSWORD_CHANGED

Purpose

Password updated.

Severity

None

IMPORTANT

Reliability

None

HIGH

Delivery

None

EMAIL mandatory

+

IN_APP

EVENT

None

LOGIN_SUCCESS

Severity

None

INFO

Reliability

None

LOW

Delivery

None

none by default

Reason:

avoid notification fatigue.

EVENT

None

SUSPICIOUS_LOGIN

Severity

None

CRITICAL

Reliability

None

VERY_HIGH

Delivery

None

EMAIL mandatory

+

TELEGRAM

+

IN_APP

15.7 CATEGORY C — CLAN

WORKFLOW EVENTS

EVENT

None

CLAN_INVITATION

Severity

None

WARNING

Reliability

None

MEDIUM

Delivery

None

Preferred Channel

+

IN_APP

EVENT

None

CLAN_JOIN_REQUEST

Severity

None

WARNING

Reliability

None

MEDIUM

Delivery

Admin:

None

EMAIL

+

TELEGRAM

EVENT

None

MEMBERSHIP_APPROVED

Severity

None

IMPORTANT

Reliability

None

HIGH

Delivery

None

EMAIL

+

Preferred Channel

+

IN_APP

EVENT

None

MEMBERSHIP_REJECTED

Severity

None

IMPORTANT

Reliability

None

HIGH

Delivery

None

EMAIL

+

IN_APP

15.8 CATEGORY D — ADMINISTRATIVE EVENTS

EVENT

None

ROLE_CHANGED

Severity

None

IMPORTANT

Reliability

None

HIGH

Delivery

None

EMAIL mandatory

+

TELEGRAM

+

IN_APP

EVENT

None

ADMIN_APPROVAL_REQUIRED

Severity

None

WARNING

Reliability

None

HIGH

Delivery

None

EMAIL

+

TELEGRAM

EVENT

None

ADMIN_ACTION_REQUIRED

Severity

None

IMPORTANT

Reliability

None

HIGH

Delivery

None

EMAIL mandatory

+

TELEGRAM

15.9 CATEGORY E — SECURITY EVENTS

EVENT

None

SECURITY_ALERT

Severity

None

CRITICAL

Reliability

None

VERY_HIGH

Delivery

None

EMAIL mandatory

+

TELEGRAM

+

IN_APP

Future Escalation

None

SMS optional

EVENT

None

ACCOUNT_LOCKED

Severity

None

CRITICAL

Reliability

None

VERY_HIGH

Delivery

None

EMAIL

+

IN_APP

EVENT

None

RECOVERY_EMAIL_CHANGED

Severity

None

CRITICAL

Reliability

None

VERY_HIGH

Delivery

None

EMAIL mandatory

15.10 CATEGORY F — ENGAGEMENT EVENTS

EVENT

None

ONBOARDING_REMINDER

Severity

None

INFO

Reliability

None

LOW

Delivery

None

Preferred Channel

EVENT

None

PROFILE_INCOMPLETE

Severity

None

WARNING

Reliability

None

MEDIUM

Delivery

None

IN_APP

+

Preferred Channel

EVENT

None

COMMUNICATION_BINDING_REMINDER

Severity

None

INFO

Reliability

None

LOW

Delivery

None

IN_APP only

Reason:

avoid spam.

15.11 Event Naming Convention

All events SHALL use:

None

UPPER_SNAKE_CASE

Pattern:

None

ENTITY_ACTION

Examples:

Correct:

None

USER_APPROVED

PASSWORD_CHANGED

ROLE_CHANGED

Forbidden:

None

approvedUser

passwordChanged

sendApprovalMessage

Doctrine Statement

None

Events describe facts.

Never implementation.

15.12 Notification Event Matrix

Event	S e v e r i t y	R e l i a b i l i t y	M a n d a t o r y	P r e f e r e d
USER_REGISTERED	I N F O	L O W	N o n e	I n - A p p
USER_APPROVED	I M P O R T A N T	H I G H	E m a i l	P r e f e r e d
PASSWORD_RESET _REQUESTED	I M	H I	E m	T e

	P O R T A N T	G H	a i l	l e g r a m
PASSWORD_CHANGED	I M P O R T A N T	H I G H	E m a i l	l i n - A p p
CLAN_INVITATION	W A R N I N G	M E D I U M	N o n e	P r e f e r r e d
ROLE_CHANGED	I M P O	H I G H	E m a	T e l e

	R T A N T		i l	g r a m
SECURITY_ALERT	C R I T I C A L	V E R Y _ H I G H	E m a i l	T e l e g r a m
ACCOUNT_LOCKED	C R I T I C A L	V E R Y _ H I G H	E m a i l	I n - A p p

15.13 Event Catalog Doctrine

Summary

EGAL-25 freezes:

EC-1

None

Events are business facts.

EC-2

None

Severity determines reliability.

EC-3

None

Reliability determines routing.

EC-4

None

Critical events require official channels.

EC-5

None

Low-value events avoid spam.

EC-6

None

Naming must stay standardized.

PART XV — FINAL EVENT

GOVERNANCE PHILOSOPHY

Core philosophy:

None

When communication becomes centralized,

events become language.

Therefore:

None

notification governance

becomes:

None

shared architectural vocabulary

for:

None

all future EGAL evolution.

PART XVI — BACKEND MODULE

DESIGN SPECIFICATION

16.1 Purpose of This Section

This section formalizes:

None

backend architecture

required for:

None

EGAL-25.x.x

Unified Communication Architecture.

The objective is:

None

implementation safety.

Without backend standardization:

the system risks:

None

communication scattered across services

hardcoded `sendEmail()`

duplicate logic

architecture drift

This section therefore freezes:

None

notification backend design.

for:

None

Node.js

Express

Prisma

Supabase PostgreSQL

architecture.

16.2 Foundational Principle

Communication SHALL become:

None

infrastructure concern.

NOT:

None

business service responsibility.

Meaning:

Business services SHALL NOT own:

None

delivery

routing

channel selection

retry

Business services only own:

None

`business truth.`

Core philosophy:

None

`Business emits intent.`

`Infrastructure delivers communication.`

16.3 Backend Module Structure

EGAL-25 introduces:

None

`notification module`

Architecture:

None

`src/`

`└─ modules/`

`└─ notifications/`

└─ orchestrator/

└─ policy/

└─ adapters/

└─ templates/

└─ queue/

└─ repository/

└─ services/

└─ jobs/

Purpose:

None

strict separation of concerns.

16.4 Module Responsibility Model

The notification architecture SHALL follow:

None

`single responsibility.`

Meaning:

each module owns:

None

`one architectural concern.`

ONLY.

16.5 Module A

Orchestrator Layer

Folder:

None

`notifications/orchestrator/`

Purpose

Acts as:

None

`central communication brain.`

Responsible for:

None

`routing decisions`

`workflow coordination`

`delivery triggering`

Responsibility

Allowed:

None

`receive notification intent`

`resolve policy`

`select channels`

`trigger adapters`

`enqueue delivery`

Forbidden:

None

business logic

role permission

UI logic

Example

Input:

TypeScript

```
emit(USER_APPROVED)
```

Orchestrator:

None

find recipients

resolve reliability

select channels

create deliveries

enqueue jobs

Example Service

TypeScript

```
notificationOrchestrator.handle(  
  
    USER_APPROVED,  
  
    userId  
  
)
```

Doctrine Statement

None

Orchestrator decides.

Adapters execute.

16.6 Module B

Policy Engine

Folder:

None

`notifications/policy/`

Purpose

Responsible for:

None

`communication decision rules.`

Questions answered:

None

`Who receives?`

`Which channels?`

`What reliability?`

`What escalation?`

Inputs

Policy evaluates:

None

role

segment

preference

accessibility profile

event severity

reliability

binding state

Example

Input:

None

PASSWORD_RESET_REQUESTED

EDITOR

preferred = Telegram

Output:

None

EMAIL mandatory

+

Telegram optional

+

In-App

Example File

None

policy/

|— routingPolicy.js

|— reliabilityPolicy.js

|— escalationPolicy.js

└— rolePolicy.js

Forbidden

Policy SHALL NOT:

None

send messages

write DB

call APIs

Doctrine Statement

None

Policy decides.

Never delivers.

16.7 Module C

Adapter Layer

Folder:

None

`notifications/adapters/`

Purpose

Responsible for:

None

`channel-specific delivery.`

Adapters SHALL remain:

None

`isolated.`

Meaning:

one adapter per:

None

`channel.`

Structure

None

`adapters/`

`email/`

`telegram/`

`zalo/`

`push/`

`inapp/`

Required Adapters

EmailAdapter

Purpose:

None

`official communication.`

TelegramAdapter

Purpose:

None

`automation-friendly delivery.`

ZaloManualAdapter

Purpose:

None

`manual-assisted workflow.`

No automation.

Only:

None

`queue`

`deep links`

`QR`

`copy templates`

WebPushAdapter

Purpose:

None

`convenience notification.`

InAppAdapter

Purpose:

None

```
canonical notification.
```

Forbidden

Adapters SHALL NOT:

None

```
contain business logic
```

```
know user role
```

```
know permissions
```

```
know workflow
```

Example

Wrong:

TypeScript

```
if (user.role === 'ADMIN')
```

inside adapter.

Correct:

Policy decides earlier.

Doctrine Statement

None

`Adapters deliver.`

`Nothing else.`

16.8 Module D

Template Engine

Folder:

None

`notifications/templates/`

Purpose

Responsible for:

None

`message generation.`

Avoids:

None

`hardcoded strings.`

Example Structure

None

`templates/`

`approval/`

`auth/`

`clan/`

`security/`

Example

Template:

None

USER_APPROVED

Email:

None

Your membership

has been approved.

Telegram:

None

🎉 Approved!

Zalo:

None

Bác đã được phê duyệt.

Benefits

Supports:

None

multi-language

multi-channel

elder wording

Doctrine Statement

None

Communication text

belongs in templates.

16.9 Module E

Queue Layer

Folder:

None

notifications/queue/

Purpose

Responsible for:

None

`asynchronous delivery.`

Business SHALL NOT wait for:

None

`communication success.`

Example Flow

Correct:

None

`Register`

`→ emit event`

`→ enqueue`

`→ continue`

Worker:

None

`send later.`

Benefits

Improves:

None

speed

retry

resilience

fault isolation

Suggested Stack

Possible options:

None

BullMQ

Redis Queue

Background Cron

Start simple.

Evolve later.

Doctrine Statement

None

Communication

must never block workflow.

16.10 Module F

Repository Layer

Folder:

None

notifications/repository/

Purpose

Responsible for:

None

database access only.

Example

Allowed:

None

```
saveNotification()
```

```
saveDelivery()
```

```
getUserPreferences()
```

Forbidden:

None

```
routing logic
```

```
message formatting
```

Doctrine Statement

None

```
Repositories persist.
```

```
Nothing else.
```

16.11 Module G

Job Workers

Folder:

None

`notifications/jobs/`

Purpose

Responsible for:

None

`background processing.`

Examples:

None

`delivery retry`

`scheduled reminder`

`failed queue recovery`

Example Jobs

None

`sendNotification.job.js`

`retryDelivery.job.js`

`cleanup.job.js`

Doctrine Statement

None

`Workers execute.`

NOT:

None

`decide.`

16.12 Backend Dependency Rules

Allowed flow:

None

`Business Service`



Notification Intent



Orchestrator



Policy



Template



Adapter



Repository

Forbidden:

None

Business Service

→ EmailAdapter

Anti-Pattern Example

Wrong:

TypeScript

```
approveUser() {  
  
    sendEmail(...)  
  
}
```

Correct:

TypeScript

```
approveUser() {  
  
    notification.emit(  
  
        USER_APPROVED  
  
    )  
  
}
```

Doctrine Statement

None

Business stays clean.

16.13 Backend Module Doctrine Summary

EGAL-25 freezes:

BM-1

None

Business emits intent.

BM-2

None

Orchestrator owns communication.

BM-3

None

Policy decides.

BM-4

None

Adapters deliver.

BM-5

None

Templates generate content.

BM-6

None

Queue prevents blocking.

BM-7

None

Repositories persist only.

BM-8

None

Workers execute.

PART XVI — FINAL BACKEND PHILOSOPHY

Core philosophy:

None

Communication infrastructure

must stay modular.

Because:

None

modular systems

survive evolution.

PART XVII — FRONTEND UX SPECIFICATION

17.1 Purpose of This Section

This section formalizes:

None

frontend communication UX

for:

None

EGAL-25.x.x

Unified Communication Architecture.

The objective is:

None

UX consistency.

Without a UX specification:

implementation risks:

None

feature creep

elder confusion

over-complex onboarding

communication anxiety

doctrine violation

This section therefore freezes:

None

user interaction rules

for:

None

communication-related UX.

The specification SHALL remain aligned with:

None

EGAL-24 Elder Support Doctrine.

Meaning:

communication features SHALL NEVER degrade:

None

low anxiety

voice support

simplicity

elder confidence.

17.2 Foundational UX Principle

Communication UX SHALL remain:

None

supportive

NOT:

None

intrusive.

The system SHALL:

None

gently suggest

NOT:

None

force setup.

Core philosophy:

None

Communication support

must feel helpful.

Never mandatory.

17.3 Communication Preference Zone

Purpose

Allow users to declare:

None

preferred communication channel.

during:

None

Register / Onboarding.

WITHOUT:

None

binding requirement.

Placement

Recommended placement:

None

`near final onboarding steps.`

NOT:

None

`first screen.`

Reason:

avoid:

None

`early complexity.`

UI Component

Suggested component:

None

`CommunicationPreferenceField`

UX Design

Single Choice (Recommended)

Reason:

elder simplicity.

Example:

None

Bác muốn hệ thống liên lạc với bác bằng cách nào?

- ☐ Zalo
- ☐ Telegram
- ☐ WhatsApp
- ☐ Email

Default Selection Logic

Recommended:

Elder mode

Default:

None

Zalo

Standard mode

Default:

None

Email

or:

None

previous behavior.

Forbidden UX

Avoid:

None

multi-checkbox setup

advanced settings

channel verification

QR immediately

during registration.

Doctrine Statement

None

Choice now.

Connection later.

17.4 Voice Guidance Integration

Communication UX SHALL integrate with:

None

EGAL-24 voice doctrine.

Communication Zone Voice

Example voice guidance:

None

Bác có thể chọn cách hệ thống liên lạc với bác.

Nếu bác hay dùng Zalo,

bác có thể chọn Zalo.

Error Speech

Errors MUST remain:

None

`submit-gated.`

Example:

Wrong:

None

`live validation spam.`

Correct:

Submit:

None

`Bác vui lòng chọn`

`một hình thức liên lạc.`

Doctrine Statement

None

Communication UX

inherits Elder Doctrine.

17.5 Deferred Communication

Reminder UX

Purpose

Later reminder:

None

connect preferred channel.

WITHOUT:

None

pressure.

Trigger Timing

Recommended timing:

After successful login

NOT:

None

immediately after registration.

Contextual moments

Example:

None

approval pending

support needed

notification missed

Reminder Design

Recommended wording:

Elder Mode

None

Bác đã chọn nhận thông báo qua Zalo.

Nếu bác muốn thuận tiện hơn,

bác có thể kết nối Zalo

với Ban Quản trị.

Việc này không bắt buộc.

Standard Mode

None

You selected Telegram

as your preferred channel.

Connect now?

Buttons

Recommended:

None

Kết nối ngay

Để sau

NOT:

None

Skip permanently

Frequency

Rule:

Never spam.

Recommended:

None

once every 7–14 days

until:

None

CONNECTED

or:

None

DISMISSED repeatedly.

Doctrine Statement

None

Gentle reminder.

Never pressure.

17.6 Zalo Manual-Assisted UX

Purpose

Provide:

None

elder-friendly communication assistance.

WITHOUT:

None

automation dependency.

UI Component

Suggested:

None

ZaloConnectDialog

Recommended Flow

Step 1

User clicks:

None

Kết nối Zalo

Step 2

Dialog opens.

Message:

None

Bác có thể liên hệ

với Ban Quản trị qua Zalo

để thuận tiện hơn.

Step 3

Options:

None

Mở Zalo

Quét mã QR

Để sau

Step 4

Prepared message:

None

Xin chào,

Tôi muốn kết nối tài khoản myClan

để nhận hỗ trợ gia phả.

Admin UX

Admin sees:

None

`manual send queue.`

Status:

None

`WAITING`

`CONNECTED`

`HELP_REQUIRED`

Doctrine Statement

None

`Humans communicate.`

`System assists.`

17.7 Telegram Connection UX

Purpose

Support:

None

automation-friendly setup.

UI Flow

Step 1

Prompt:

None

Nhận thông báo qua Telegram?

Step 2

Button:

None

Kết nối Telegram

Step 3

Deep link:

None

Open @myClan_bot

Step 4

User:

None

/start

Step 5

System confirms:

None

Telegram đã được kết nối.

Doctrine Statement

None

Automation

must feel simple.

17.8 Notification Center UX

Purpose

Provide:

None

`canonical notification history.`

Because:

None

`in-app notification`

`is source of truth.`

UI Component

Suggested:

None

`NotificationCenter`

Access

Recommended:

None

bell icon

in:

None

top navigation.

Categories

Examples:

None

Thông báo

Bảo mật

Gia phả

Quản trị

Visual Priority

CRITICAL:

None

high visibility.

INFO:

None

lightweight.

Elder Mode

Recommended:

None

large text

simple labels

voice support

Doctrine Statement

None

Notification history

must feel reassuring.

17.9 Notification Fatigue Prevention UX

The system SHALL avoid:

None

communication overload.

Rules:

Rule 1

Never show:

None

multiple reminders

simultaneously.

Rule 2

Do NOT repeatedly ask:

None

Connect Zalo?

daily.

Rule 3

Avoid:

None

`modal overload.`

Prefer:

None

`soft card reminder.`

Rule 4

Critical communication MAY interrupt.

Examples:

None

`security warning`

`account compromise`

Doctrine Statement

None

Preserve trust.

Avoid annoyance.

17.10 UX Components Inventory

Recommended components:

None

CommunicationPreferenceField

NotificationCenter

NotificationBell

ZaloConnectDialog

TelegramConnectDialog

ReminderCard

CommunicationStatusBadge

State Labels

Recommended wording:

Connected

None

Đã kết nối

Not Connected

None

Chưa kết nối

Optional

None

Không bắt buộc

Later

None

Để sau

Avoid wording:

None

Required immediately

Mandatory now

Incomplete account

Doctrine Statement

None

Words shape confidence.

17.11 UX Governance Rules

UX-1

None

Never force communication setup.

UX-2

None

Communication setup

must remain dismissible.

UX-3

None

Voice support

must remain available.

UX-4

None

Elder users

always get reassurance wording.

UX-5

None

Notification UX

must remain simple.

UX-6

None

Critical alerts

override convenience rules.

17.12 Frontend UX Doctrine

Summary

EGAL-25 freezes:

UX-1

None

Choice now.

Connection later.

UX-2

None

Gentle reminder.

Never pressure.

UX-3

None

Communication UX

inherits Elder Doctrine.

UX-4

None

Humans communicate.

System assists.

UX-5

None

Avoid notification fatigue.

UX-6

None

Confidence

always beats complexity.

PART XVII — FINAL UX PHILOSOPHY

Core philosophy:

None

Good communication UX

feels supportive.

Not technical.

Therefore:

EGAL-25 communication SHALL remain:

None

`simple`

`elder-friendly`

`reassuring`

`optional-first.`

PART XVIII — IMPLEMENTATION

SPRINT PLAN

18.1 Purpose of This Section

This section formalizes:

None

`practical implementation roadmap`

for:

None

`EGAL-25.x.x`

`Unified Communication Architecture.`

The objective is:

None

controlled execution.

Without a sprint plan:

implementation risks:

None

architecture drift

random coding

premature optimization

UX breakage

technical debt

This section therefore freezes:

None

implementation sequencing.

aligned with:

None

EGAL doctrines

Q1/Q2

controlled evolution philosophy.

18.2 Foundational Principle

EGAL-25 SHALL evolve:

None

incrementally.

NOT:

None

through rewrite.

Every sprint SHALL preserve:

Q1

None

Do not break

working business flow.

Q2

None

Preserve UX/UI

unless intentionally changed.

Q3

None

Prefer anchor-level patch.

Q4

None

Communication never blocks onboarding.

Core Philosophy

None

Stable systems

evolve carefully.

18.3 Sprint Overview

Implementation SHALL proceed through:

None

6 controlled sprints.

Timeline:

S p r i n t	Goal	Risk
S p r i n t 2 5 .0	Foundation & Schema	LO W
S p r i n t 2 5	Communication Preference UX	LO W

.
1

S
p
r
i
n
t

2
5

.
2

S
p
r
i
n
t

2
5

.
3

S
p
r
i
n
t
2

Notification Persistence

LO
W-
ME
DIU
M

Telegram Integration

ME
DIU
M

Zalo Manual-Assisted

LO
W

5

.

4

S

Notification Center & UX

LO

p

Polish

W

r

i

n

t

2

5

.

5

18.4 SPRINT 25.0

Foundation & Schema

Objective

Establish:

None

communication infrastructure.

WITHOUT:

None

visible user change.

Scope

Implement:

None

Prisma schema

database migration

backend skeleton

based on:

None

PART XIV

PART XVI

Database Work

Create:

None

UserCommunicationPreference

CommunicationBinding

Notification

NotificationRecipient

NotificationDelivery

Enums:

None

ChannelType

ReliabilityLevel

BindingState

DeliveryStatus

NotificationEvent

Backend Work

Create module skeleton:

None

`notifications/`

Folders:

None

`orchestrator/`

`policy/`

`adapters/`

`templates/`

`queue/`

`repository/`

`jobs/`

No logic yet.

Frontend Work

None.

No UI changes.

UAT Criteria

System MUST:

None

behave exactly

like EGAL-24.

No regression.

Risk

None

LOW

Rollback Strategy

Simply:

None

disable new module.

Doctrine Statement

None

Build invisible foundation first.

18.5 SPRINT 25.1

Communication Preference UX

Objective

Introduce:

None

preferred communication selection

during:

None

Register / Onboarding.

WITHOUT:

None

channel setup.

Scope

Frontend only.

Files Likely Affected

Examples:

None

RegisterForm

JoinClanForm

CreateClanForm

WaitingPage

AuthPage

UX Work

Add:

None

Communication Preference Zone

Options:

None

Zalo

Telegram

WhatsApp

Email

Single choice only.

Voice Guidance

Integrate:

None

ZoneVoiceButton

Example wording:

None

Bác có thể chọn

cách hệ thống liên lạc với bác.

Backend Work

Persist:

None

UserCommunicationPreference.

No binding.

UAT Criteria

Users MUST:

None

complete onboarding

without confusion.

No increase in:

None

drop-off.

Risk

None

LOW

Rollback Strategy

Hide field.

Doctrine Statement

None

Choice first.

No pressure.

18.6 SPRINT 25.2

Notification Persistence

Objective

Introduce:

None

canonical notification infrastructure.

WITHOUT:

None

`external communication.`

Scope

Backend heavy.

Backend Work

Implement:

None

`Notification persistence`

`NotificationRecipient`

`Delivery logging`

Business services SHALL emit:

None

`events.`

NOT:

None

`sendEmail()`

yet.

Example Migration

Before:

None

`ForgotPassword`

`→ send email`

After:

None

`ForgotPassword`

`→ emit(PASSWORD_RESET_REQUESTED)`

Still:

None

`email fallback allowed.`

temporarily.

Frontend Work

Optional:

None

`NotificationBell placeholder.`

UAT Criteria

All notifications MUST:

None

`exist in database.`

Even if:

None

`delivery fails.`

Risk

None

`LOW-MEDIUM`

Rollback

Temporary:

None

`email-first fallback.`

Doctrine Statement

None

`Remember communication.`

18.7 SPRINT 25.3

Telegram Automation

Objective

Introduce:

None

`real multi-channel delivery.`

through:

None

`Telegram.`

Scope

Telegram only.

No Zalo yet.

Backend Work

Create:

None

TelegramAdapter

Implement:

None

chat_id binding

bot workflow

delivery logging

Frontend Work

Add:

None

Connect Telegram

inside:

None

Communication Preferences.

UX Flow

None

Connect Telegram

→ Open Bot

→ /start

→ connected

UAT Criteria

Telegram MUST:

None

deliver successfully.

for:

None

approval

password reset

security alert.

Risk

None

MEDIUM

Rollback

Disable:

None

TelegramAdapter.

Email remains.

Doctrine Statement

None

Automate stable platforms first.

18.8 SPRINT 25.4

Zalo Manual-Assisted

Objective

Support:

None

elder communication reality.

WITHOUT:

None

automation dependency.

Scope

Introduce:

None

manual-assisted flow.

Frontend Work

Add:

None

ZaloConnectDialog

Options:

None

Mở Zalo

Quét mã QR

Để sau

Backend Work

Add:

None

ZaloManualAdapter

Only:

None

manual queue

copy templates

deep links

No automation.

Admin UX

Create:

None

Manual Queue View

Statuses:

None

WAITING

CONNECTED

HELP_REQUIRED

UAT Criteria

Elder users MUST:

None

understand flow

without assistance.

Risk

None

LOW

Rollback

Disable:

None

manual queue UI.

Doctrine Statement

None

Support people.

`Do not automate fragility.`

18.9 SPRINT 25.5

Notification Center & UX Polish

Objective

Provide:

None

`complete communication experience.`

Frontend Work

Create:

None

`NotificationCenter`

`NotificationBell`

ReminderCard

CommunicationStatusBadge

UX Work

Add:

None

soft reminder logic

fatigue prevention

dismiss behavior

Elder UX

Add:

None

large text

voice support

reassurance wording

UAT Criteria

Notification experience MUST feel:

None

`simple`

`non-intrusive`

`elder-friendly.`

Risk

None

`LOW`

Rollback

Disable:

None

`NotificationCenter.`

Core communication remains.

Doctrine Statement

None

Communication should feel reassuring.

18.10 Sprint Governance Rules

Rule SP-1

None

No big bang rewrite.

Rule SP-2

None

Stable business flow

always protected.

Rule SP-3

None

Backend before UX.

Rule SP-4

None

Infrastructure first.

Rule SP-5

None

Elder UX never regresses.

Rule SP-6

None

Communication

must remain optional-first.

Rule SP-7

None

Every sprint

must remain independently deployable.

18.11 Final Sprint Summary

EGAL-25 implementation SHALL remain:

None

controlled

incremental

reversible

while preserving:

None

working EGAL-24 system.

PART XVIII — FINAL IMPLEMENTATION PHILOSOPHY

Core philosophy:

None

Good architecture

is implemented safely.

Therefore:

EGAL-25 chooses:

None

small safe steps

instead of:

None

large risky transformation.

PART XVIII — IMPLEMENTATION

SPRINT PLAN

18.1 Purpose of This Section

This section formalizes:

None

practical implementation roadmap

for:

None

EGAL-25.x.x

Unified Communication Architecture.

The objective is:

None

controlled execution.

Without a sprint plan:

implementation risks:

None

architecture drift

random coding

premature optimization

UX breakage

technical debt

This section therefore freezes:

None

implementation sequencing.

aligned with:

None

EGAL doctrines

Q1/Q2

controlled evolution philosophy.

18.2 Foundational Principle

EGAL-25 SHALL evolve:

None

incrementally.

NOT:

None

through rewrite.

Every sprint SHALL preserve:

Q1

None

Do not break

working business flow.

Q2

None

Preserve UX/UI

unless intentionally changed.

Q3

None

Prefer anchor-level patch.

Q4

None

Communication never blocks onboarding.

Core Philosophy

None

Stable systems

evolve carefully.

18.3 Sprint Overview

Implementation SHALL proceed through:

None

6 controlled sprints.

Timeline:

Sprint	Goal	Risk
Sprint 2.5.0	Foundation & Schema	LOW

S
p
r
i
n
t
2
5
.
1

Communication
Preference UX

LO
W

S
p
r
i
n
t
2
5
.
2

Notification Persistence

LO
W–
ME
DIU
M

S
p
r
i
n
t
2
5
.
3

Telegram Integration

ME
DIU
M

S	Zalo Manual-Assisted	LO
p		W
r		
i		
n		
t		
2		
5		
.		
4		

S	Notification Center & UX	LO
p	Polish	W
r		
i		
n		
t		
2		
5		
.		
5		

18.4 SPRINT 25.0

Foundation & Schema

Objective

Establish:

None

communication infrastructure.

WITHOUT:

None

visible user change.

Scope

Implement:

None

Prisma schema

database migration

backend skeleton

based on:

None

PART XIV

PART XVI

Database Work

Create:

None

UserCommunicationPreference

CommunicationBinding

Notification

NotificationRecipient

NotificationDelivery

Enums:

None

ChannelType

ReliabilityLevel

BindingState

DeliveryStatus

NotificationEvent

Backend Work

Create module skeleton:

None

`notifications/`

Folders:

None

`orchestrator/`

`policy/`

`adapters/`

`templates/`

`queue/`

`repository/`

`jobs/`

No logic yet.

Frontend Work

None.

No UI changes.

UAT Criteria

System MUST:

None

behave exactly

like EGAL-24.

No regression.

Risk

None

LOW

Rollback Strategy

Simply:

None

disable new module.

Doctrine Statement

None

Build invisible foundation first.

18.5 SPRINT 25.1

Communication Preference UX

Objective

Introduce:

None

preferred communication selection

during:

None

Register / Onboarding.

WITHOUT:

None

channel setup.

Scope

Frontend only.

Files Likely Affected

Examples:

None

RegisterForm

JoinClanForm

CreateClanForm

WaitingPage

AuthPage

UX Work

Add:

None

Communication Preference Zone

Options:

None

Zalo

Telegram

WhatsApp

Email

Single choice only.

Voice Guidance

Integrate:

None

ZoneVoiceButton

Example wording:

None

Bác có thể chọn

cách hệ thống liên lạc với bác.

Backend Work

Persist:

None

`UserCommunicationPreference.`

No binding.

UAT Criteria

Users MUST:

None

`complete onboarding`

`without confusion.`

No increase in:

None

`drop-off.`

Risk

None

`LOW`

Rollback Strategy

Hide field.

Doctrine Statement

None

Choice first.

No pressure.

18.6 SPRINT 25.2

Notification Persistence

Objective

Introduce:

None

canonical notification infrastructure.

WITHOUT:

None

`external communication.`

Scope

Backend heavy.

Backend Work

Implement:

None

`Notification persistence`

`NotificationRecipient`

`Delivery logging`

Business services SHALL emit:

None

`events.`

NOT:

None

```
sendEmail()
```

yet.

Example Migration

Before:

None

```
ForgotPassword
```

```
→ send email
```

After:

None

```
ForgotPassword
```

```
→ emit(PASSWORD_RESET_REQUESTED)
```

Still:

None

```
email fallback allowed.
```

temporarily.

Frontend Work

Optional:

None

`NotificationBell placeholder.`

UAT Criteria

All notifications MUST:

None

`exist in database.`

Even if:

None

`delivery fails.`

Risk

None

`LOW-MEDIUM`

Rollback

Temporary:

None

`email-first fallback.`

Doctrine Statement

None

`Remember communication.`

18.7 SPRINT 25.3

Telegram Automation

Objective

Introduce:

None

`real multi-channel delivery.`

through:

None

Telegram.

Scope

Telegram only.

No Zalo yet.

Backend Work

Create:

None

TelegramAdapter

Implement:

None

chat_id binding

bot workflow

delivery logging

Frontend Work

Add:

None

Connect Telegram

inside:

None

Communication Preferences.

UX Flow

None

Connect Telegram

→ Open Bot

→ /start

→ connected

UAT Criteria

Telegram MUST:

None

deliver successfully.

for:

None

approval

password reset

security alert.

Risk

None

MEDIUM

Rollback

Disable:

None

TelegramAdapter.

Email remains.

Doctrine Statement

None

`Automate stable platforms first.`

18.8 SPRINT 25.4

Zalo Manual-Assisted

Objective

Support:

None

`elder communication reality.`

WITHOUT:

None

`automation dependency.`

Scope

Introduce:

None

manual-assisted flow.

Frontend Work

Add:

None

ZaloConnectDialog

Options:

None

Mở Zalo

Quét mã QR

Để sau

Backend Work

Add:

None

ZaloManualAdapter

Only:

None

manual queue

copy templates

deep links

No automation.

Admin UX

Create:

None

Manual Queue View

Statuses:

None

WAITING

CONNECTED

HELP_REQUIRED

UAT Criteria

Elder users MUST:

None

understand flow

without assistance.

Risk

None

LOW

Rollback

Disable:

None

manual queue UI.

Doctrine Statement

None

Support people.

Do not automate fragility.

18.9 SPRINT 25.5

Notification Center & UX Polish

Objective

Provide:

None

complete communication experience.

Frontend Work

Create:

None

NotificationCenter

`NotificationBell`

`ReminderCard`

`CommunicationStatusBadge`

UX Work

Add:

None

`soft reminder logic`

`fatigue prevention`

`dismiss behavior`

Elder UX

Add:

None

`large text`

`voice support`

reassurance wording

UAT Criteria

Notification experience MUST feel:

None

simple

non-intrusive

elder-friendly.

Risk

None

LOW

Rollback

Disable:

None

NotificationCenter.

Core communication remains.

Doctrine Statement

None

Communication should feel reassuring.

18.10 Sprint Governance Rules

Rule SP-1

None

No big bang rewrite.

Rule SP-2

None

Stable business flow

always protected.

Rule SP-3

None

Backend before UX.

Rule SP-4

None

Infrastructure first.

Rule SP-5

None

Elder UX never regresses.

Rule SP-6

None

Communication

must remain optional-first.

Rule SP-7

None

Every sprint

must remain independently deployable.

18.11 Final Sprint Summary

EGAL-25 implementation SHALL remain:

None

controlled

incremental

reversible

while preserving:

None

working EGAL-24 system.

PART XVIII — FINAL

IMPLEMENTATION PHILOSOPHY

Core philosophy:

None

Good architecture

is implemented safely.

Therefore:

EGAL-25 chooses:

None

small safe steps

instead of:

None

large risky transformation.

APPENDIX — ARCHITECTURE

GLOSSARY & TERMINOLOGY

Purpose of This Appendix

This appendix freezes:

None

shared architectural vocabulary.

for:

None

EGAL-25.x.x

Unified Communication Architecture.

Purpose:

prevent:

None

misunderstanding

inconsistent implementation

terminology drift

All terms below SHALL be treated as:

None

canonical definitions.

for:

None

architecture

backend

frontend

UAT

documentation.

A

Accessibility Profile

Definition

A user characteristic that determines:

None

interaction experience.

Examples:

None

ELDER_MODE

VOICE_SUPPORT

LARGE_UI

STANDARD_MODE

Accessibility profile affects:

None

UX

voice guidance

wording

interaction complexity

Accessibility profile SHALL NOT affect:

None

authorization.

Adapter

Definition

A:

None

`channel-specific delivery module.`

Responsible only for:

None

`message delivery.`

Examples:

None

`EmailAdapter`

`TelegramAdapter`

`ZaloManualAdapter`

`WebPushAdapter`

`InAppAdapter`

Adapters SHALL NOT contain:

None

`business logic`

`permission logic`

`workflow decisions`

Canonical principle:

None

`Adapters deliver.`

`Nothing else.`

B

Binding

Definition

Represents:

None

`actual communication connection.`

Examples:

Telegram:

None

`chat_id linked`

Zalo:

None

manual relationship established

Binding is:

None

optional.

Binding $\neq$ preference.

Binding State

Definition

Lifecycle of communication connection.

States:

None

DECLARED

SUGGESTED

CONNECTED

DISABLED

Meaning:

DECLARED

User selected preference.

No connection.

SUGGESTED

System suggested setup.

CONNECTED

Channel active.

DISABLED

Connection removed.

C

Canonical Notification

Definition

A notification that exists:

None

`inside myClan.`

Meaning:

every communication MUST first become:

None

in-app record.

before:

None

external delivery.

Canonical notification becomes:

None

source of truth.

Channel

Definition

A communication medium.

Examples:

None

EMAIL

TELEGRAM

ZALO_PZ

WHATSAPP

WEB_PUSH

IN_APP

Channels SHALL remain:

None

delivery adapters.

NOT:

None

identity providers.

Communication Binding

Definition

A persistent record of:

None

channel connection state.

Example:

None

Telegram connected

vs.

None

Zalo preferred only.

Communication Preference

Definition

A declared user preference.

Example:

None

Preferred:

Zalo

WITHOUT:

None

requiring connection.

Canonical principle:

None

Preference first.

Connection later.

Convenience Channel

Definition

A channel optimized for:

None

ease

habit

engagement.

Examples:

None

ZALO

TELEGRAM

WHATSAPP

WEB_PUSH

Convenience channels SHALL NOT replace:

None

official communication.

D

Deferred Communication Binding

Definition

The doctrine where:

None

channel setup

happens later.

NOT:

None

during onboarding.

Goal:

None

low anxiety UX.

Canonical principle:

None

Choice now.

Connection later.

Delivery

Definition

The act of:

None

sending communication.

Examples:

None

email sent

telegram sent

zalo manual required

Delivery SHALL remain:

None

observable.

Delivery Status

Definition

State of communication execution.

Examples:

None

PENDING

SENT

FAILED

MANUAL_REQUIRED

READ

E

Event

Definition

A:

None

`business fact.`

Examples:

None

`USER_APPROVED`

`PASSWORD_CHANGED`

`ROLE_CHANGED`

`SECURITY_ALERT`

Events describe:

None

`what happened.`

NOT:

None

`how to communicate.`

Canonical principle:

None

Events describe facts.

Event Catalog

Definition

The official inventory of:

None

business events.

Each event defines:

None

severity

reliability

routing

mandatory channels

I

In-App Notification

Definition

Communication stored:

None

`inside myClan.`

Acts as:

None

`canonical communication history.`

Every notification SHALL exist:

None

`in-app first.`

Intent

Definition

A communication abstraction created from:

None

`business event.`

Example:

Event:

None

`USER_APPROVED`

Intent:

None

`Notify approved user.`

Intent answers:

None

`what communication`

`should happen.`

M

Mandatory Channel

Definition

A channel REQUIRED for:

None

`minimum reliability.`

Example:

None

EMAIL

for:

None

password reset.

Mandatory channels SHALL override:

None

preference.

when required.

Manual-Assisted Channel

Definition

A channel supported through:

None

human assistance.

instead of:

None

full automation.

Example:

None

ZALO_PZ

Supported by:

None

QR

deep links

prepared messages

manual queue

N

Notification

Definition

A persistent communication record.

Independent from:

None

channel.

A notification represents:

None

communication intent.

NOT:

None

delivery.

Canonical principle:

None

Notification $\neq$ Delivery.

Notification Center

Definition

Frontend UI displaying:

None

notification history.

Acts as:

None

source of reassurance.

Especially for:

None

elder users.

Notification Orchestrator

Definition

The:

None

central communication engine.

Responsible for:

None

policy resolution

routing

delivery coordination

retry

Canonical principle:

None

Business emits intent.

Orchestrator handles delivery.

O

Official Channel

Definition

A channel with:

None

high reliability.

Example:

None

EMAIL.

Used for:

None

security

recovery

important workflow.

Opportunistic Channel

Definition

Convenience-only delivery.

Examples:

None

WEB_PUSH

IN_APP

Failure remains:

None

acceptable.

P

Policy Engine

Definition

Responsible for:

None

communication decisions.

Questions answered:

None

who

what channel

what reliability

what escalation

Policy SHALL NOT:

None

deliver communication.

Preferred Channel

Definition

User-selected communication preference.

Examples:

None

ZALO

TELEGRAM

WHATSAPP

Preferred channel MAY influence routing.

But SHALL NOT override:

None

mandatory reliability.

Q

Queue

Definition

Background processing layer.

Responsible for:

None

asynchronous delivery.

Purpose:

prevent:

None

communication blocking business flow.

Canonical principle:

None

Communication

must never block workflow.

R

Reliability

Definition

Required communication certainty.

Levels:

None

LOW

MEDIUM

HIGH

VERY_HIGH

Higher reliability:

None

more redundancy.

Reliability Level

Definition

Minimum communication requirement.

Examples:

None

LOW = best effort

HIGH = official delivery

VERY_HIGH = certainty first

Reminder Fatigue

Definition

Negative UX caused by:

None

too many notifications.

EGAL-25 SHALL prevent:

None

notification spam.

S

Severity

Definition

Business importance classification.

Levels:

None

INFO

WARNING

IMPORTANT

CRITICAL

Severity determines:

None

`minimum reliability.`

Source of Truth

Definition

The authoritative system record.

For communication:

None

`in-app notification`

becomes:

None

`canonical history.`

U

Unified Communication Layer (UCL)

Definition

The architectural layer introduced by:

None

EGAL-25.

Responsible for:

None

communication orchestration.

Includes:

None

notification

policy

delivery

queue

adapters

V

Voice Guidance

Definition

Accessibility assistance through:

None

speech support.

Inherited from:

None

EGAL-24 Elder Support Doctrine.

Communication UX SHALL integrate with:

None

voice assistance.

FINAL GLOSSARY PRINCIPLE

All terminology in:

None

EGAL-25.x.x

SHALL use:

None

these canonical meanings.

to prevent:

None

implementation ambiguity

team misunderstanding

architecture drift.

Core philosophy:

None

Shared vocabulary

creates shared architecture.